

Coach C#

<Atelier 1/> Créer sa première application

Sommaire

1	INTRODUCTION.....	3
1.1	CONTEXTE FONCTIONNEL.....	3
1.2	CONTEXTE TECHNIQUE.....	4
2	COMMENCER A PROGRAMMER C#.....	5
2.1	DEVELOPPER UN PREMIER PROGRAMME C#.....	5
2.2	COMPRENDRE LA COMPILATION .NET	12
3	CREER SA PREMIERE SOLUTION AVEC VISUAL STUDIO.....	20
3.1	ORGANISER LA SOLUTION.....	20
3.2	APPRIVOISER L'ENVIRONNEMENT DE DEVELOPPEMENT	31
4	ENRICHIR LE COMPORTEMENT DE L'APPLICATION	36
4.1	MODIFIER LA FONCTION D'AJOUT	36
4.2	AJOUTER DES OBJETS SYMPATHIQUES.....	46
5	POUR ALLER PLUS LOIN... ..	52
5.1	LES LIENS UTILES.....	52
5.2	IDEE/RESSOURCE	52

1 Introduction

Cet atelier s'inscrit dans le cadre du tutorial du coach C# dont l'objectif est la découverte et l'utilisation du langage C# avec Visual Studio 2012 pour la construction d'applications avec une approche orientée objet.

Le langage C# a été annoncé en 2000, lors de la grande conférence des développeurs Microsoft à Orlando (PDC – Professional Developers Conference).

L'objectif de C# est de proposer un langage simple, complètement orienté objet, sécurisé, et assurant le développement rapide et élégant d'applications possédant de hautes performances :

- C# est simple, car il possède relativement peu de mots clés ;
- C# est un langage objet. C'est tellement vrai que tout dans C# est objet ☺ ;
- C# est sécurisé, car c'est un langage fortement typé, c'est-à-dire que tous les types sont systématiquement vérifiés par le compilateur pour vous aider à détecter des éventuels bugs le plus tôt possible dans le processus de développement. Dans ce tutorial, lorsqu'une fonction liée à la sécurisation est illustrée, vous verrez le petit logo en marge ;
- C# aide au développement rapide et élégant des applications. Les programmes devenant de plus en plus complexes et longs à écrire, il y a deux façons de réduire les temps de développement : soit le développeur apprend à se servir rapidement d'un clavier pour taper plus de mots à la minute, soit le nombre de mots nécessaires pour l'expression d'un besoin est réduit. C# propose des constructions succinctes, et très élégantes (certes, c'est un point de vue personnel) pour exprimer un besoin. Dans ce tutorial, lorsqu'un groupe de lignes de code peut être exprimé d'une manière concise et/ou élégante, vous verrez le petit logo en marge ;



Avec C#, vous pouvez construire tout type d'applications : Windows, Web, des services, etc. Dans le cadre de ce tutorial, nous allons créer une application Windows, mais uniquement en support d'illustration des points abordés.

Les exercices de l'atelier 1 se proposent de vous présenter les toutes premières étapes pour comprendre ce qu'il y a derrière la création d'une application C#, et de vous familiariser avec l'environnement .Net de Microsoft.

1.1 Contexte fonctionnel

Rappel du contexte fonctionnel du tutorial du coach C#

L'objectif du tutorial du Coach C# est d'accompagner les développeurs à la prise en main du langage C# dans le cadre de projets de développement. Ce tutorial se concentre donc sur le langage, en l'abordant plus du point de vue des besoins des développeurs que de celui d'une liste exhaustive des caractéristiques techniques.

Ce que nous ne verrons pas dans le tutorial du Coach C#

L'objectif de ce tutorial n'est pas d'expliquer :

- Comment accéder à des bases de données ;
- Les pratiques de développement des applications Windows ou Web ;

- L'ensemble des objets fournis par le Framework ;
- La méthodologie de gestion des projets de développement ;

Mais nous allons ensemble voir pourquoi C# s'impose aujourd'hui comme le langage de développement dans le monde Microsoft.

Contexte fonctionnel du premier atelier

L'atelier décrit les réponses aux premières questions que se pose tout développeur abordant C# et .net pour la première fois : Qu'est-ce qu'un programme C# ? Où est .net ? Comment est-il compilé ? Pourquoi utiliser Visual Studio 2012 ? Quels outils et utilitaires utiliser dans les développements ?

Vous allez donc créer votre première application C#, qui lit des informations à partir d'un fichier.

1.2 Contexte technique

A la fin de cet atelier, vous saurez comment :

- Coder une application avec le bloc-notes ;
- Localiser les principaux éléments de .Net ;
- Compiler et protéger votre application ;
- Développer une solution avec Visual Studio C# Express ;
- Localiser les objets du Framework.

La solution de cet atelier est disponible dans le répertoire **..\Atelier 1\Solution**. Les fichiers utiles, auxquels font référence les exercices sont disponibles dans le répertoire **..\Atelier 1\Fichiers utiles**.

2 Commencer à programmer C#

Dans cet exercice, vous allez apprendre à :

- Ecrire un premier programme C# ;
- Compiler le programme avec le compilateur csc.exe ;
- Signer et distribuer les assemblages ;
- Documenter le code programmé.

Objectif

L'objectif de cet exercice est de positionner les différents éléments .Net liés à la programmation C#.

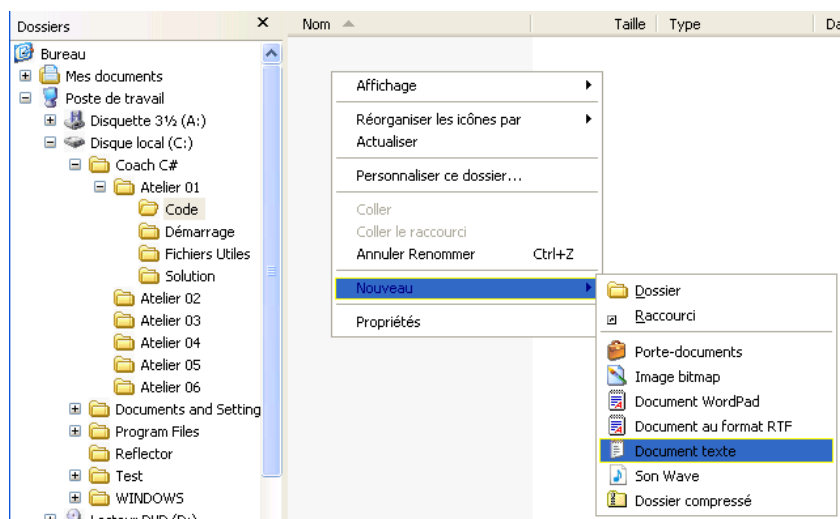
Contexte fonctionnel

Le programme développé est un calculateur simple en mode console.

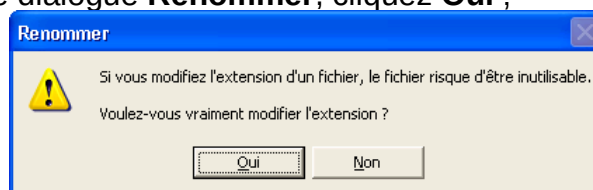
2.1 Développer un premier programme C#

Déroulement de l'exercice :

1. Créez un premier programme :
 - Ouvrez l'**Explorateur Windows** ;
 - Sélectionnez un répertoire de travail (par exemple **C:\Coach\Atelier 01\Code**) ;
 - Faites un clic droit pour créer un nouveau fichier texte ;



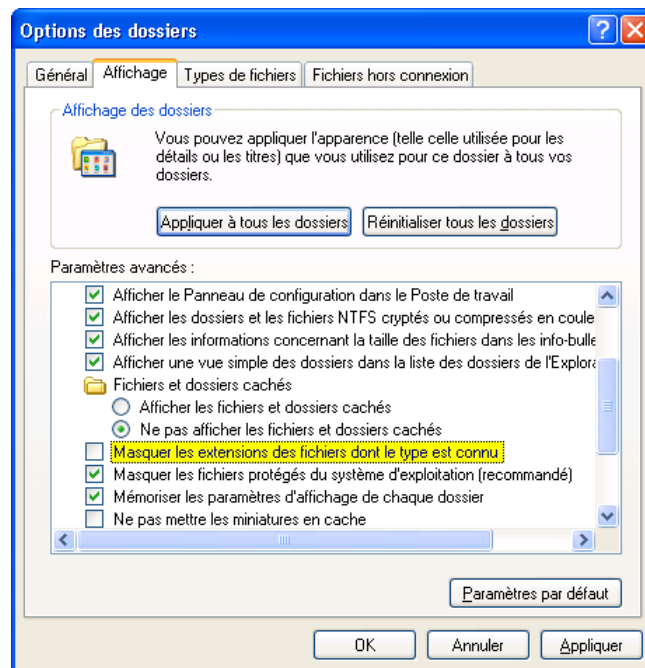
- Renommez le fichier en **PremierProgramme.cs** ;
- Sur la boîte de dialogue **Renommer**, cliquez **Oui** ;



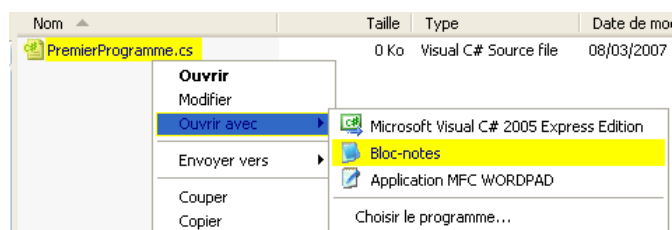


Si la **boîte** de dialogue **Renommer** n'apparaît pas, c'est certainement parce que dans le paramétrage d'affichage des dossiers, l'option **Masquer les extensions des fichiers dont le type est connu** est activée. Pour désactiver cette option :

- Sélectionnez le menu **Outils > Options des dossiers...** de l'explorateur Windows ;
- Sélectionnez l'onglet **Affichage** ;
- Dans les **Paramètres avancés**, utilisez la barre de défilement pour visualiser l'option **Masquer les extensions des fichiers dont le type est connu** ;
- Désélectionnez cette option.



- Faites un clic droit sur **PremierProgramme.cs**, et sélectionnez l'option **Ouvrir avec > Bloc-notes** :



- Ajouter le code suivant :

Code C#

```
namespace Coach
{
    class programme
    {
        static void Main()
        {
            System.Console.WriteLine("Bonjour à tous");
            System.Console.ReadKey();
        }
    }
}
```

Espace de
nommage

Les accolades { } indiquent le
début et la fin de chaque bloc.

C'est le point d'entrée du programme

Le point virgule indique la fin de la ligne

- Sauvegardez votre programme par les menus du Bloc-notes ;

Et voilà, vous venez de faire votre premier programme C# ! Super.



La principale difficulté lorsque l'on apprend un langage, est qu'il faudrait apprendre tout le langage avant d'apprendre quelque chose. En effet, même ce simple programme d'affichage en mode console d'une chaîne de caractères utilise de nombreuses fonctionnalités de C# que nous allons découvrir dans ce tutorial. Mais pas de soucis, nous sommes tous passé par le même chemin.



La première chose à savoir quand on écrit du C#, c'est que le langage est sensible à la casse. Aussi les deux lignes suivantes ne sont pas les mêmes pour le compilateur C# :

```
System.Console.WriteLine("Bonjour à tous");
```

```
system.Console.WriteLine("Bonjour à tous");
```

L'une fonctionne, et l'autre va donner une erreur de compilation (mais laquelle est-ce ?)



La première ligne du programme est `namespace Coach`. Durant ce tutorial, vous allez créer de nombreux noms d'objets ou de types. Pour éviter les conflits avec des noms déjà existants, **l'espace de nommage** précise à quoi se rattache le nom. C'est un peu le nom de famille des objets que vous allez créer. Par exemple, la classe `programme` a comme nom de famille `Coach`. Son nom complet est donc **`Coach.Programme`**.



L'utilisation des accolades ouvrantes et fermantes indique au compilateur les blocs du programme. Un bloc, c'est un peu comme un paragraphe de texte, qui regrouperait des phrases traitant d'un même sujet. Un bloc peut donc contenir plusieurs lignes. En programmation, les blocs structurent les programmes, et sont fondamentaux pour définir la portée d'un élément.



Comme tout programme exécutable, on doit indiquer par où commencer les choses. C'est l'objectif du bloc `static void Main()` qui contient systématiquement les premiers ordres à exécuter lors du lancement.



En français, les phrases finissent toutes par un point. En C#, elles finissent toutes par un point-virgule. C'est très utile, car ainsi vous allez pouvoir exprimer une même phrase sur plusieurs lignes de code source, sans gêner le compilateur. Dans le cas de programme complexe, cela aide fortement à la compréhension (et donc à la maintenance !). Par exemple, les deux lignes de code suivantes sont similaires et ne génèrent aucune erreur de compilation :

```
System.Console.WriteLine("Bonjour à tous");

System
    .Console
    .WriteLine
    ("Bonjour à tous")
    ;
```

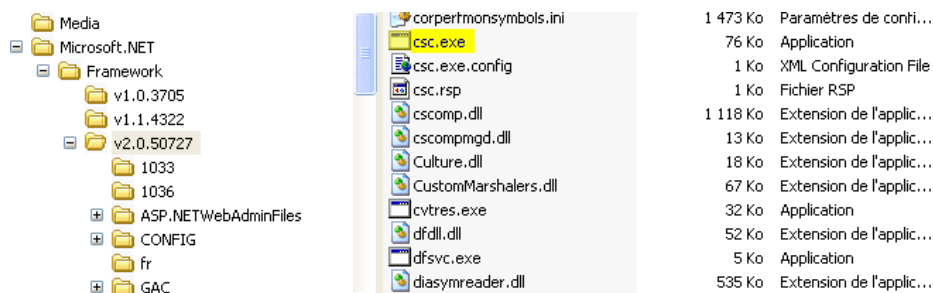


Quand on vient d'un langage de programmation comme VB, les principales difficultés du départ pour écrire les codes sont la casse des noms, les accolades et le point-virgule. Comme on dit, un homme averti en vaut deux ...

2. Compilez le programme :



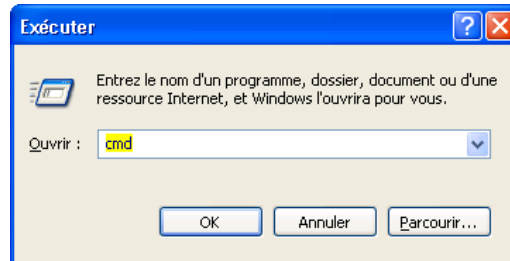
L'installation du Framework .Net 3.5 a créée un nouveau répertoire **Microsoft .NET\Framework** sous **c:\WINDOWS**. Ce sous répertoire contient le(s) Framework(s) .Net installé(s). La version 3.5 est installée dans le sous répertoire **v3.5**, et contient tout ce qu'il faut pour développer et compiler sur .Net, notamment le programme **csc.exe** qui est le compilateur C#.



Si vous travaillez sur une machine possédant le Framework .Net 3.0 (livré en standard avec Vista, par exemple), vous avez un sous répertoire supplémentaire **v3.0**. La version 3.0 fonctionnant au-dessus de la version 2.0, ce répertoire ne contient que les briques supplémentaires fournies par la version 3.0 (Windows Presentation Foundation, Windows Communication Foundation, et Windows Workflow Foundation). Le

compilateur C# est donc dans le répertoire de la version 2.0, même sous Vista.

- Cliquez le menu **Démarrer > Exécuter** ;
- Dans la **boîte** de dialogue, taper l'ordre **cmd** ;



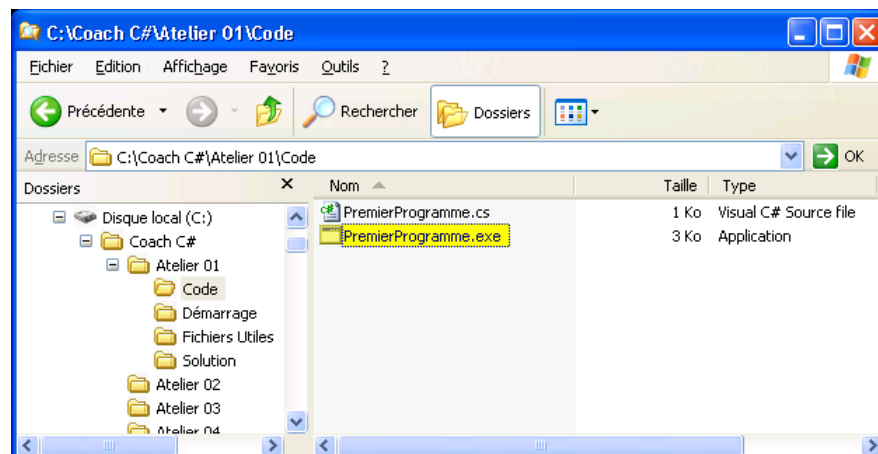
- A l'intérieur de la nouvelle invite de commande, positionnez-vous dans le sous répertoire contenant votre programme – dans notre cas, tapez l'ordre **cd C:\Coach C#\Atelier 01\Code** ;
- Indiquez un chemin d'exécution vers le répertoire contenant le compilateur C#, en tapant l'ordre suivant :
Path C:\WINDOWS\Microsoft.NET\Framework\v3.5
- Compilez votre programme avec l'ordre **csc premierprogramme.cs** ;

```
C:\WINDOWS\system32\cmd.exe
C:\Coach C#\Atelier 01\Code>path C:\WINDOWS\Microsoft.NET\Framework\v2.0.50727
C:\Coach C#\Atelier 01\Code>csc premierprogramme.cs
Microsoft (R) Visual C# 2005 Compiler version 8.00.50727.42
pour Microsoft (R) Windows (R) 2005 Framework version 2.0.50727
Copyright (C) Microsoft Corporation 2001-2005. Tous droits réservés.

C:\Coach C#\Atelier 01\Code>_
```

S'il existe une erreur de codage, le compilateur vous l'indique. Il vous reste à corriger la ligne ☺. Laissez l'invite de commande ouverte.

- Avec l'explorateur Windows, naviguez jusqu'à votre sous répertoire de travail (par exemple **C:\Coach C#\Atelier 01\Code**) ; Un nouveau fichier **PremierProgramme.exe** a été généré par le compilateur.



- Double-cliquez le programme **PremierProgramme.exe**, et une nouvelle console s'affiche, en vous souhaitant Bonjour !



Lors de la compilation, nous avons laissé l'ensemble des options par défaut. C'est pour cela qu'un fichier d'extension `.exe` a été généré. Pour voir l'ensemble des options disponibles avec le compilateur, tapez l'ordre `csc /?`.

3. Compilez une librairie :



Il arrive souvent que des codes complexes nécessitent plusieurs fichiers différents pour exprimer des besoins différents, ou que des fonctions soient partagées par plusieurs programmes. Dans ce cas, une bonne pratique est de regrouper ces fonctions partagées dans des librairies qui seront compilées et utilisées par tous les programmes qui en ont besoin.

- Dans votre répertoire de travail (par exemple **C:\Coach C#\Atelier 01\Code**), créez un nouveau programme **Calculateur.cs**, en utilisant le Bloc-notes ;
- En utilisant le Bloc-notes, ajoutez le code suivant, qui ajoute un calculateur capable de faire une addition ;

Code C#

```
namespace Coach
{
    public class Calculateur
    {
        public static int Ajouter(int Valeur01, int Valeur02)
        {
            return Valeur01 + Valeur02;
        }
    }
}
```

Espace de nommage est le même que précédemment

Le calculateur ajoute deux valeurs entières (int)



Le mot clé `public` indique que l'élément concerné sera visible de l'extérieur. Nous reviendrons plus en détail sur la visibilité lors de ce tutorial.

- Sauvegardez ce nouveau programme.

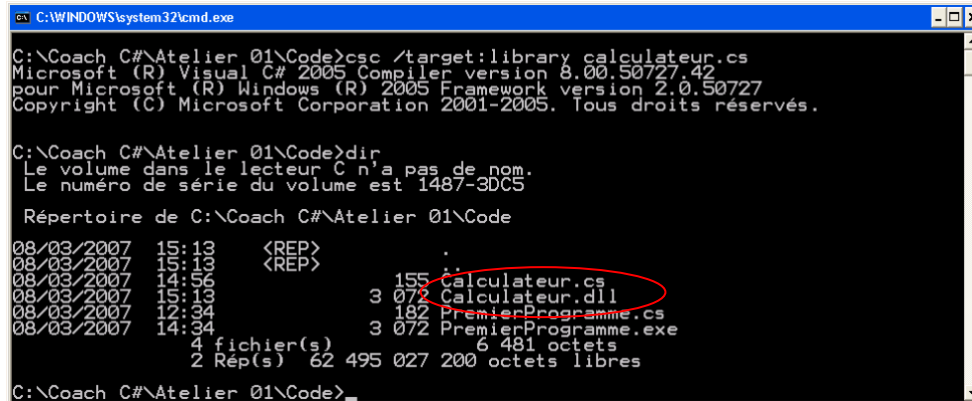


Ce programme ne contient pas de bloc `static void Main()`. En effet, c'est une librairie qui va être appelée par un exécutable extérieur, et donc elle n'a pas besoin d'avoir de point d'entrée. Les librairies possèdent une extension `.dll`.

- Avec l'invite de commande précédemment ouverte, compilez la librairie en indiquant que vous souhaitez obtenir un fichier avec une extension `.dll`, en utilisant l'ordre suivant : **`csc /target:library calculateur.cs`**.

S'il existe une erreur de codage, le compilateur vous l'indique. Il vous reste à corriger la ligne ☺. Laissez l'invite de commande ouverte.

- Dans l'invite de commande, tapez l'ordre **dir** pour afficher le contenu du répertoire. Le fichier **Calculateur.dll** a été généré.



```
C:\WINDOWS\system32\cmd.exe

C:\Coach C#\Atelier 01\Code>csc /target:library calculateur.cs
Microsoft (R) Visual C# 2005 Compiler version 8.00.50727.42
pour Microsoft (R) Windows (R) 2005 Framework version 2.0.50727
Copyright (C) Microsoft Corporation 2001-2005. Tous droits réservés.

C:\Coach C#\Atelier 01\Code>dir
Le volume dans le lecteur C n'a pas de nom.
Le numéro de série du volume est 1487-3DC5

Répertoire de C:\Coach C#\Atelier 01\Code

08/03/2007 15:13 <REP> .
08/03/2007 15:13 <REP> ..
08/03/2007 14:56 155 calculateur.cs
08/03/2007 14:56 3 072 calculateur.dll
08/03/2007 14:56 182 PremierProgramme.cs
08/03/2007 14:56 3 072 PremierProgramme.exe
08/03/2007 14:56 6 481 octets
4 fichier(s) 3 072 PremierProgramme.exe
2 Rép(s) 62 495 027 200 octets libres

C:\Coach C#\Atelier 01\Code>
```

4. Utilisez la librairie de calcul :

- En utilisant le Bloc-notes, ouvrez le fichier **PremierProgramme.cs** ;
- Modifiez le code initial de façon à utiliser le calculateur ;

Code C#

```
namespace Coach
{
    class programme
    {
        static void Main()
        {
            System.Console.WriteLine("Bonjour à tous");

            System.Console.
                WriteLine(
                    "L'addition de 10 et 5 est : {0}.",
                    Calculateur.Ajouter(10,5)
                );

            System.Console.ReadKey();
        }
    }
}
```



La commande est ici écrite sur plusieurs lignes afin d'améliorer la lisibilité dans le cadre de ce document. Vous pouvez bien sûr l'écrire en une seule ligne, sans oublier le point-virgule de fin.

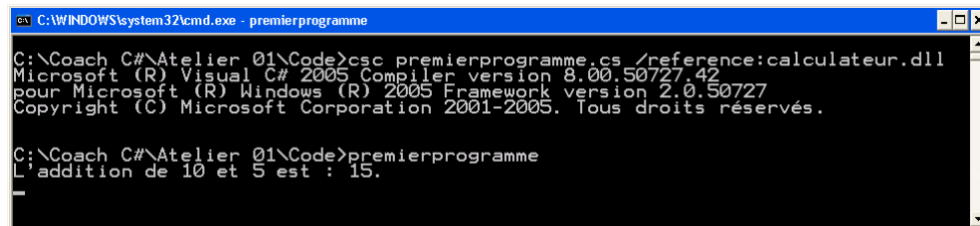


La commande utilise les possibilités de formatage des chaînes de caractères en passant les paramètres entre accolades ; Plus d'informations sur le formatage des chaînes est disponible sur les liens :

<http://msdn2.microsoft.com/fr-fr/netframework/aa569608.aspx>

[http://msdn2.microsoft.com/fr-fr/library/b1csw23d\(VS.80\).aspx](http://msdn2.microsoft.com/fr-fr/library/b1csw23d(VS.80).aspx)

- Sauvegardez le programme ;
- A partir de l'invite de commande, compilez de nouveau **PremierProgramme.cs**, mais en indiquant maintenant que vous voulez référencer la librairie qui contient le calculateur ; L'ordre à taper est le suivant : **csc premierprogramme.cs /reference:calculateur.dll** ;
- A partir de l'invite de commande, tapez **premierprogramme** ; le résultat du calcul s'affiche.



```
C:\WINDOWS\system32\cmd.exe - premierprogramme
C:\Coach C#\Atelier 01\Code>csc premierprogramme.cs /reference:calculateur.dll
Microsoft (R) Visual C# 2005 Compiler version 8.00.50727.42
pour Microsoft (R) Windows (R) 2005 Framework version 2.0.50727
Copyright (C) Microsoft Corporation 2001-2005. Tous droits réservés.

C:\Coach C#\Atelier 01\Code>premierprogramme
L'addition de 10 et 5 est : 15.
```

Et voilà, vous venez de créer votre premier programme C# utilisant un composant, ce dernier pourrait d'ailleurs être partagé par plusieurs programmes. Félicitations !

2.2 Comprendre la compilation .Net

Il s'agit dans cet exercice de regarder ce que le compilateur C# génère, et comment ce code agit avec le Framework .Net.

Déroulement de l'exercice :

1. Regardez ce qui a été compilé :



Lors d'une compilation C#, le compilateur convertit le code source en langage MSIL (Microsoft Intermediate Language), un jeu d'instructions indépendant du processeur qui est converti ensuite en code natif. MSIL inclut des instructions pour le chargement, le stockage, l'initialisation et l'appel de méthodes sur des objets, ainsi que des instructions pour la réalisation d'opérations arithmétiques et logiques, le flux de contrôle, l'accès direct à la mémoire, la gestion des exceptions et d'autres opérations. Avant d'exécuter du code, vous devez d'abord convertir le langage MSIL en code spécifique du processeur, généralement à l'aide du compilateur JIT (Just-In-Time) fourni par le Framework .Net dans le Common Language Runtime (CLR). On dit alors que le code est géré par le CLR, d'où le nom de *code managé*.



Lorsqu'un compilateur produit un code MSIL, il génère aussi des métadonnées. Les métadonnées décrivent les types contenus dans votre code, y compris la définition de chaque type, les signatures des membres de chaque type, les membres référencés par votre code, et d'autres données que le runtime utilise au moment de l'exécution. Le code MSIL et les métadonnées sont stockés dans un fichier exécutable portable (PE)

qui est basé sur le fichier Microsoft PE publié qu'il prolonge et sur le format COFF (Common Object File Format) utilisé traditionnellement pour le contenu exécutable. Ce format de fichier, qui accepte le code MSIL ou le code natif ainsi que les métadonnées, permet au système d'exploitation de reconnaître les images du Common Language Runtime. La présence de métadonnées dans le fichier en même temps que le jeu d'instructions MSIL permet à votre code de se décrire lui-même, ce qui signifie que les bibliothèques de types et IDL (Interface Definition Language) ne sont pas nécessaires. Le runtime recherche les métadonnées dans le fichier et les extrait selon les besoins, au moment de l'exécution.



Chaque exécutable ou librairie compilé spécifiquement pour .Net possède donc ses métadonnées et son manifeste. C'est pour cette raison qu'il n'y a plus besoin d'enregistrer les composants .Net dans la base de registre de l'ordinateur ! Et c'est aussi pour cela que la cohabitation de plusieurs versions du même composant est possible sur une même machine.

Exemple de Code MSIL

```
.method public hidebysig static int32 Ajouter(int32
Valeur01, int32 Valeur02) cil managed
{
    // Code size          9 (0x9)
    .maxstack 2
    .locals init (int32 V_0)
    IL_0000: nop
    IL_0001: ldarg.0
    IL_0002: ldarg.1
    IL_0003: add mul
    IL_0004: stloc.0
    IL_0005: br.s          IL_0007

    IL_0007: ldloc.0
    IL_0008: ret
} // end of method Calculateur::Ajouter
```



Comme vous venez de le voir, le compilateur C# transforme le code C# en MSIL, puis le code est *assemblé* dans un fichier d'extension .dll. C'est pour cette raison que nous appelons les librairies compilées avec .Net « assemblage » (ou « assembly » en littérature anglaise).

2. Partagez votre assemblage avec d'autres programmes :



Pour la plupart des applications .NET Framework, vous localisez les assemblages qui composent votre application dans le répertoire de l'application. Il est néanmoins possible de les mettre ailleurs, soit dans un sous répertoire de l'application (en paramétrant ce chemin dans la

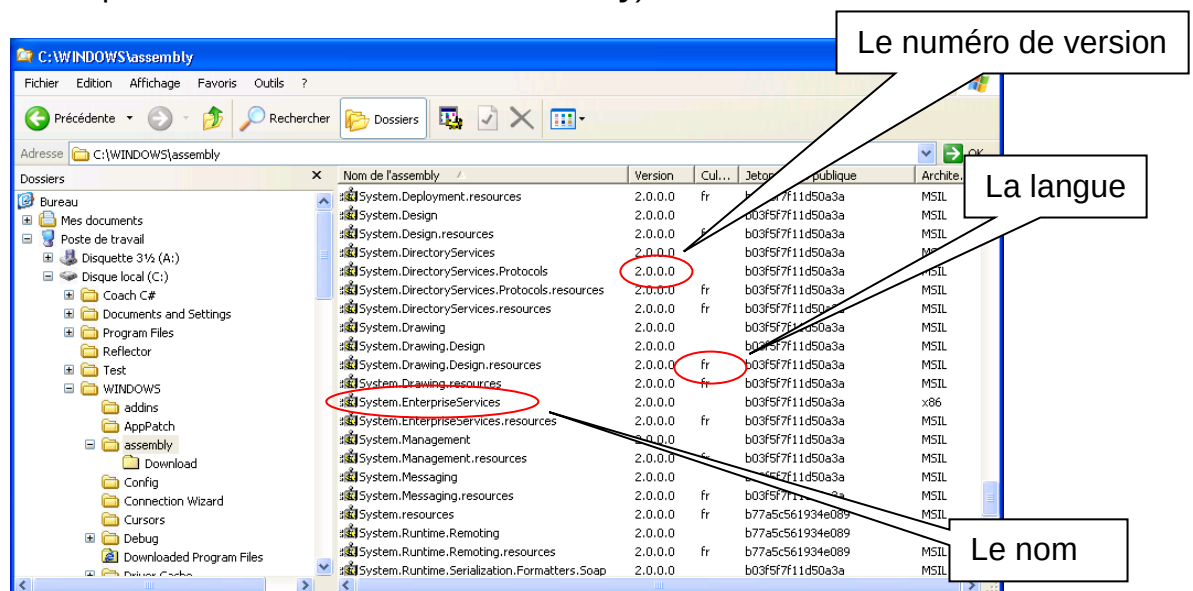
configuration de l'application), soit dans un référentiel centralisé contenant tous les assemblages partagés installés sur votre poste de travail. Ce référentiel centralisé est le **Global Assembly Cache** ou **GAC**.



Chaque ordinateur sur lequel le Common Language Runtime (CLR) est installé possède un cache de code à l'échelle de l'ordinateur appelé **Global Assembly Cache** (GAC). Le GAC stocke les assemblages spécialement destinés à être partagés entre plusieurs applications sur l'ordinateur. Vous ne devez partager des assemblages en les installant dans le GAC qu'en cas de nécessité. En règle générale, vous devez garder les dépendances vers les assemblages privés et rechercher les assemblages dans le répertoire de l'application. En outre, il n'est pas nécessaire d'installer des assemblages dans le GAC pour les rendre accessibles à COM Interop ou au code non managé.



Le GAC est sur le disque de votre ordinateur, dans le répertoire **assembly** sous le répertoire d'installation de Windows (donc, classiquement dans **c:\Windows\assembly**).



Le GAC n'est pas un répertoire comme les autres, car il peut contenir des assemblages ayant un même nom, mais des numéros de versions différentes.



Les assemblages déployés dans le GAC doivent avoir un nom fort. Lorsqu'un assemblage est ajouté au GAC, des contrôles d'intégrité sont effectués sur tous les fichiers qui composent l'assemblage. Le cache effectue ces contrôles d'intégrité pour vérifier qu'aucun assemblage n'a été falsifié, par exemple, lorsqu'un fichier a été modifié, mais que le manifeste ne reflète pas cette modification.



Pour ajouter un assemblage dans le GAC, vous pouvez :

- Utiliser l'**Explorateur Windows** pour faire glisser des assemblages dans le GAC ;



Pour retirer un assemblage dans le GAC, vous pouvez :

- Utiliser l'**Explorateur Windows** pour faire un clic droit sur l'assemblage à retirer, en sélectionnant le menu **Désinstaller** ;



3. Fournissez le code de vos programmes :



Chaque langage .Net respecte le CLI (Common Language Infrastructure), et basé sur ce standard, les compilateurs génèrent du MSIL. Il est ainsi possible, en repartant du MSIL, de générer un pseudo-code C# (ou dans n'importe quel langage .Net d'ailleurs). Autrement dit, quand vous livrez vos applications, vous livrez aussi le pseudo-code de vos programmes. Pour peu que vous oubliiez les fichiers d'extension .pdb (informations de débogage), vous livrez alors la totalité de votre code source !

4. Documentez le code de vos programmes :



Comme la plupart des développeurs, vous avez la directive de documenter vos codes. Mais vous ne devez pas échapper à la règle : comme la plupart des développeurs, la documentation n'est pas votre tasse de thé. Le C# va vous aider à faire une bonne documentation, sans pénaliser votre principale fonction : développer.



C# propose quatre catégories de commentaires, que vous pouvez utiliser dans vos développements :

- La ligne simple de commentaire, précédée par **//**
- Les commentaires délimités, encadrés par **/* votre commentaire */**
- La ligne simple de documentation, précédée par **///**
- Et enfin, la documentation délimitée, qui combine les deux possibilités précédentes ;

L'intérêt de la ligne simple de documentation et de la documentation délimitée est qu'au moment de la compilation du code, il est possible de préciser que l'on souhaite l'extraction de la documentation. Un fichier XML est alors généré, et il suffit d'y appliquer une feuille de style XSL-T pour obtenir la documentation de votre code.

- En utilisant le Bloc-notes, ouvrez le fichier **Calculateur.cs** ;
- Modifiez le code de façon à ajouter les commentaires et la documentation ;

Code C#

```
/*
Auteur : Coach C#
Commentaire : Programme de calcul avancé
```

Révisions et Modifications

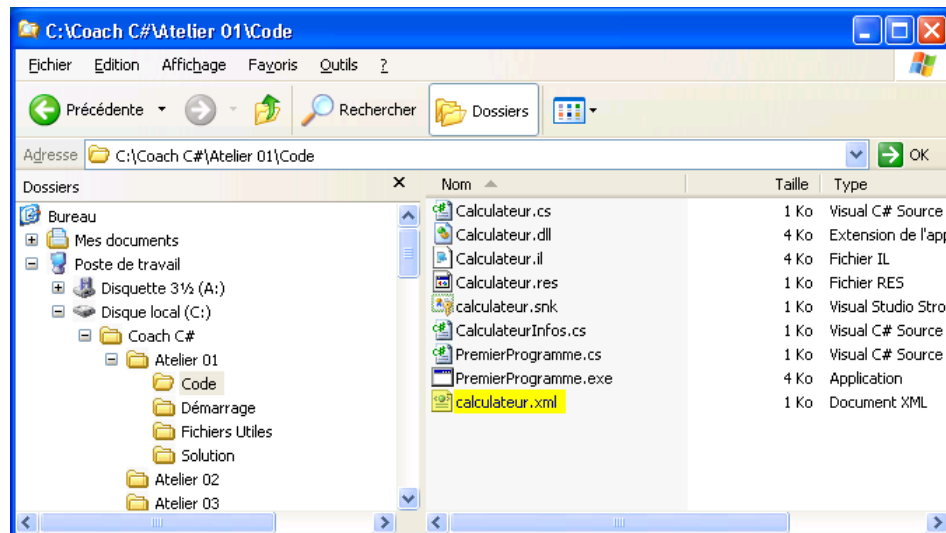
Date	Par	Commentaires
03/03/2007	Coach	Version Initiale

***/**

```
namespace Coach
{
    /// <summary>
    /// Cette classe implémente les fonctions de calcul
    /// </summary>
    public class Calculateur
    {
        /// <summary>
        /// Cette fonction ajoute deux valeurs entières
        /// </summary>
        /// <param name="Valeur01">Première valeur</param>
        /// <param name="Valeur02">Deuxième valeur</param>
        /// <returns>l'entier de résultat</returns>
        public static int Ajouter(int Valeur01, int Valeur02)
        {
            return Valeur01 + Valeur02; // retour valeur
        }
    }
}
```

Avec toutes ces balises de documentation, il devrait y avoir dans vos codes maintenant plus de lignes de commentaires que de lignes de programme !

- Sauvegardez votre fichier ;
- Dans l'invite de commande, compilez le code C# du calculateur, en référençant maintenant les informations de l'assemblage et en précisant que vous souhaitez extraire la documentation, en tapant l'ordre :
csc /target:library calculateur.cs /doc:calculateur.xml
- Avec l'explorateur Windows, naviguer jusqu'à votre sous répertoire de travail (par exemple **C:\Coach\Atelier 01\Code**) ; Un nouveau fichier **calculateur.xml** a été généré par le compilateur. Ce fichier contient la documentation de votre code au format XML ;



Dans le cas o   une des balises de documentation n'est pas coh  rente avec le code (passage de param  tres par exemple), le compilateur va vous indiquer un avertissement lors de la compilation, pour vous aider    maintenir la documentation    niveau avec le code   

- Faites un **double clic** sur votre fichier **calculateur.xml** ; Il s'ouvre dans votre navigateur Internet. Mais c'est vrai que ce n'est pas trop lisible. On va maintenant le rendre plus facile d'utilisation ;
- Faites un copier/coller du fichier **documentation.xsl**,    partir du r  pertoire contenant les fichiers utiles de l'atelier (par exemple **C:\Coach\Atelier 01\Fichiers Utiles\Exercice 01**),    destination de votre r  pertoire de travail (par exemple **C:\Coach\Atelier 01\Code**) ;
- En utilisant le Bloc-notes, ouvrez le fichier **calculateur.xml** ;
- Juste en dessous de la balise de d  finition **<?xml**, ajoutez la ligne de r  f  rencement de la feuille de style de transformation :
<?xml-stylesheet type="text/xsl" href="documentation.xsl"?>

Code XML de calculateur.xml

```
<?xml version="1.0"?>
<?xml-stylesheet type="text/xsl" href="Documentation.xsl"?>
<doc>
  <assembly>
    <name>Calculateur</name>
  </assembly>
  <members>
    <member name="T:Coach.Calculateur">
      <summary>
        Cette classe impl  mente les fonctions de calcul
      </summary>
    </member>
    <member name="M:Coach.Calculateur.Ajouter(System.Int32,System.Int32)">
      <summary>
        Cette fonction ajoute deux valeurs enti  res
      </summary>
      <param name="Valeur01">Premi  re valeur</param>
      <param name="Valeur02">Deuxi  me valeur</param>
      <returns>l'entier de r  sultat</returns>
    </member>
  </members>
</doc>
```

Pr  fixe de type

```
</member>
</members>
</doc>
```

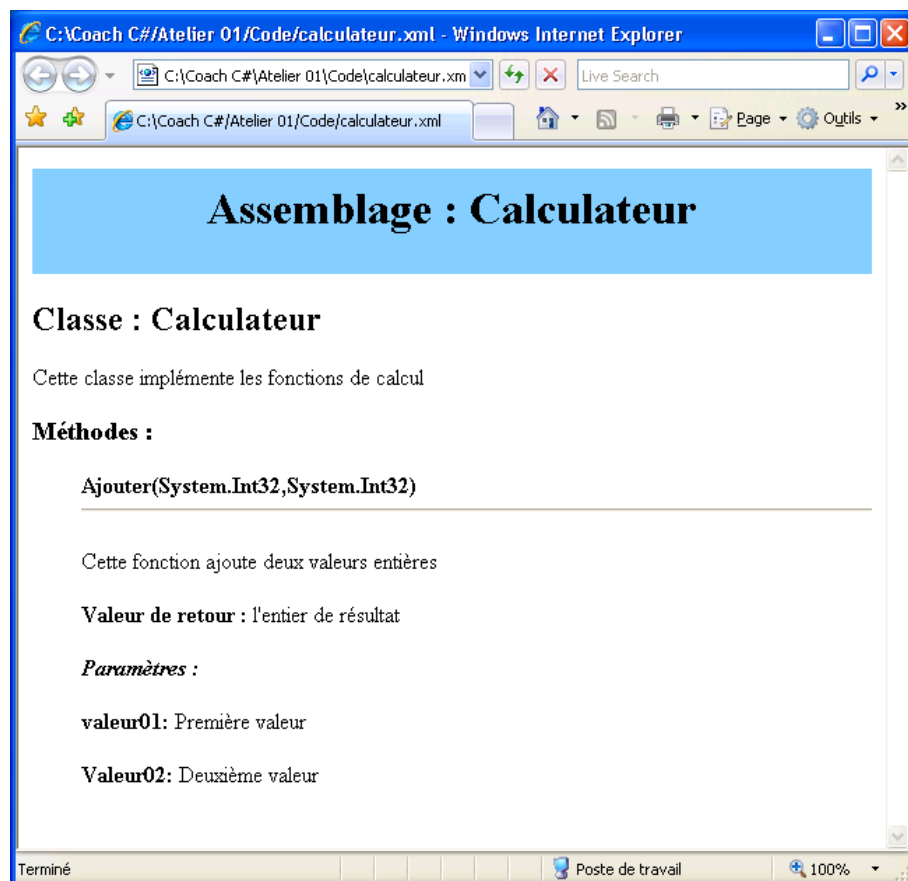
- Sauvegardez le fichier **calculateur.xml** ;



Le générateur de documentation assigne des ID aux noms des éléments. T est le préfixe des types, par exemple. Voici la liste des principaux préfixes :

Préfixe	Elément
E	Événement (Event)
F	Champ (Field)
M	Méthode (Method)
N	Espace de noms (Namespace)
P	Propriété (Property)
T	Type
!	Erreur

- Faites un **double clic** sur votre fichier **calculateur.xml** ; Il s'ouvre dans votre navigateur Internet, avec une présentation plus lisible ;



La feuille de style fournie en exemple est simple, mais vous pouvez la modifier pour l'adapter à vos besoins. Il est aussi possible de faire un petit outil Windows ou un petit site web de visualisation des fichiers de documentation, en association automatiquement le fichier XML avec la

feuille de style XSL.



Un article (en anglais) de juin 2002 mais toujours d'actualité (la feuille de style de l'atelier s'en inspire) :

<http://msdn.microsoft.com/msdnmag/issues/02/06/XMLC/>



La documentation du SDK MS-XML :

<http://msdn2.microsoft.com/fr-fr/library/ms760399.aspx>



La liste des balises recommandées pour les commentaires de documentation :

[http://msdn2.microsoft.com/fr-fr/library/5ast78ax\(VS.80\).aspx](http://msdn2.microsoft.com/fr-fr/library/5ast78ax(VS.80).aspx)

3 Créer sa première solution avec Visual Studio

Dans cet exercice, vous allez apprendre à :

- Créer une solution avec Microsoft Visual Studio ;
- Utiliser les outils de l'IDE et gérer les extraits de code ;
- Déboguer votre application.

Objectif

L'objectif de cet exercice est de découvrir l'environnement de développement intégré (IDE) de Visual Studio.

Contexte fonctionnel

Comme vous l'avez constaté durant le premier exercice, s'il reste possible de développer en utilisant un éditeur de texte tel que le Bloc-notes, le codage est fortement simplifié par un bon environnement de développement. Cela tombe bien, Microsoft fournit Visual Studio, dont la version C# Express est utilisable sans avoir à payer de licence. Nous allons donc continuer le développement de notre application, en créant une solution contenant deux projets pour gérer nos fichiers de code, avec cette version de l'IDE.



Pour bien comprendre la différence entre une solution et un projet, chaque projet développé avec l'IDE va donner un assemblage .Net avec l'extension .dll ou .exe. Dans le cadre d'un développement, il arrive souvent que la compilation doive donner plusieurs assemblages, comme dans le cas de l'exercice précédent où nous avons un fichier .exe et un fichier .dll. Une solution va contenir des codes de projets différents mais relatifs à une même problématique de codage. Une solution contient donc au minimum un projet.

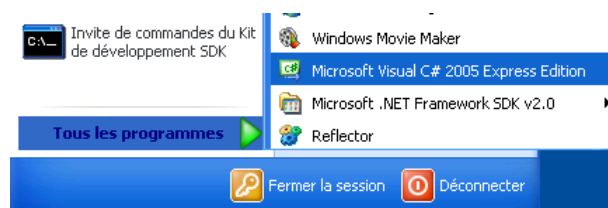


Il est important de comprendre que l'environnement de développement n'enrichit ni le langage, ni ce que peut faire la plateforme .Net. Par contre, il est conçu pour fournir des outils d'aide au développement. Autrement dit, la différence entre les différentes versions de Visual Studio réside dans les outils, modèles et options disponibles en fonction des versions.

3.1 Organiser la solution

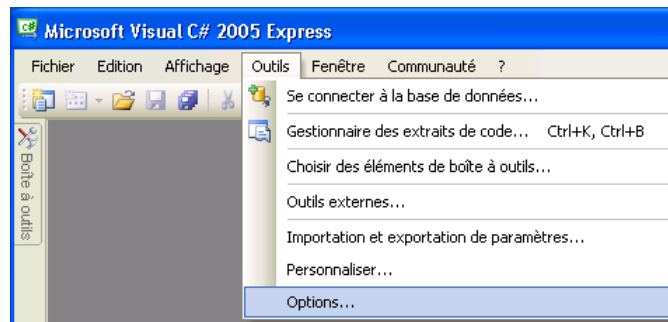
Déroulement de l'exercice :

1. Paramétrez Visual C# Express:
 - Lancez **Visual Studio** à partir du menu **Démarrer > Tous les programmes > Microsoft Visual Studio** ;

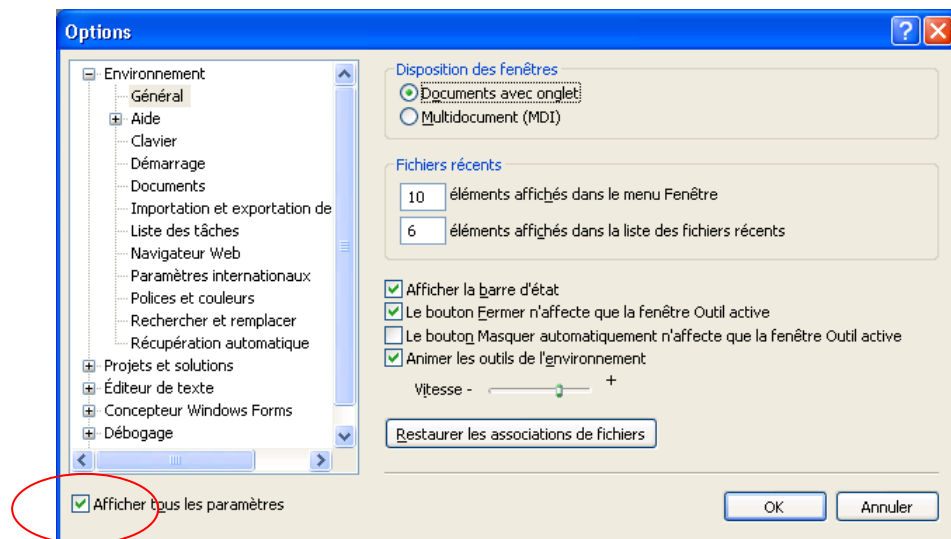


Visual Studio s'affiche à votre écran ;

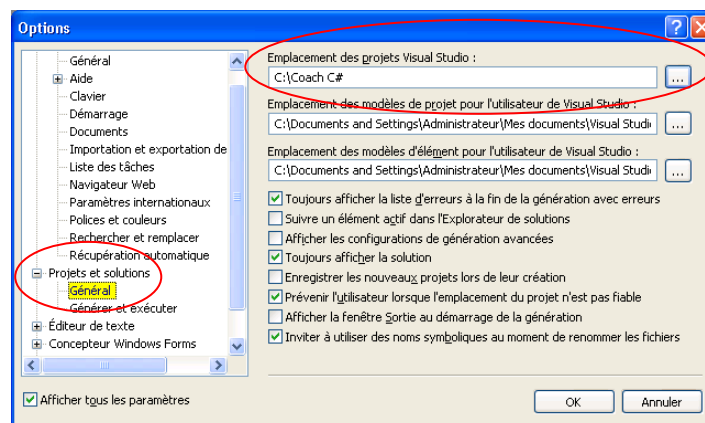
- Cliquez le menu **Outils > Option ...** de Visual Studio ;



- Dans la boîte de dialogue **Option**, cochez **Afficher tous les paramètres** ;



- Dans l'arborescence des options, cliquez sur **Projet et solutions > Général** ;
- Pour l'option **Emplacement des projets Visual Studio**, indiquez le chemin où se trouvent vos codes (par exemple **C:\Coach C#**), en utilisant le bouton  ;



- Cliquez le bouton **OK** ;



En changeant l'option **Emplacement des projets Visual Studio**, vous venez d'indiquer le chemin de sauvegarde par défaut des différents projets que vous allez créer.



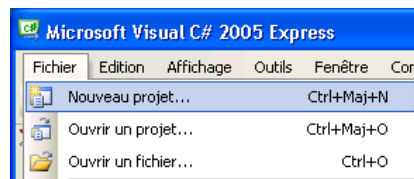
La boîte de dialogue des options présente plusieurs dizaines de paramètres que vous pouvez changer afin de modifier le fonctionnement de Visual C# 2005 Express.

2. Créez la solution :

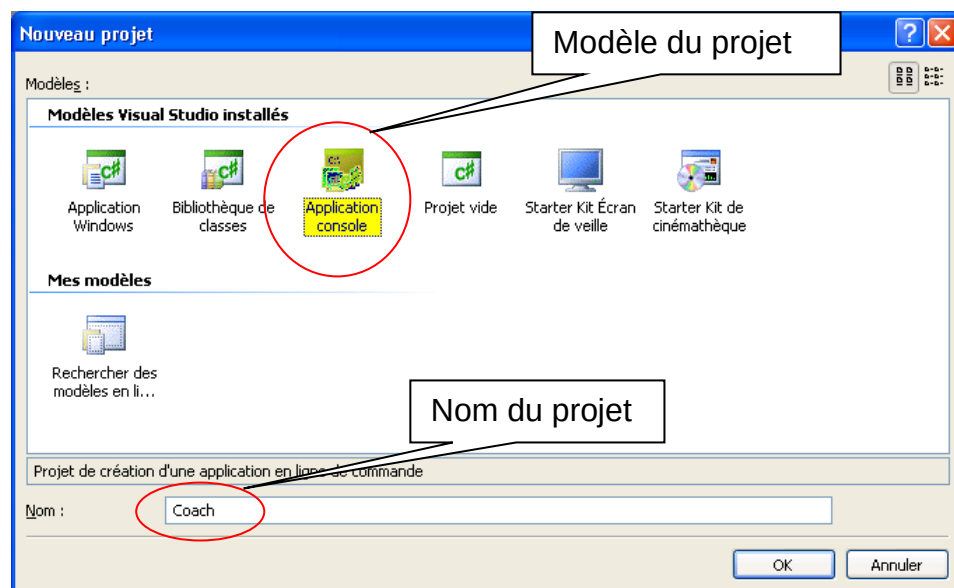


La solution contient dans notre exercice deux projets : le projet correspondant à notre fichier .exe, et le projet contenant la librairie de calcul .dll.

- Dans Visual C# Express, cliquez le menu **Fichier > Nouveau projet ...**

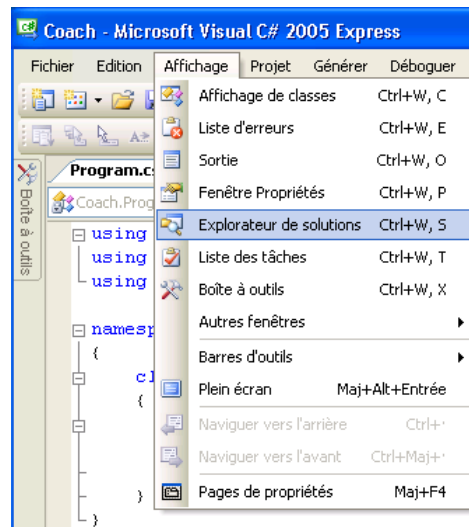


- La **boîte** de dialogue **Nouveau Projet** s'affiche. Dans cette **boîte** de dialogue, sélectionnez le modèle de projet **Application console** et indiquez un nom de projet **Coach** ;

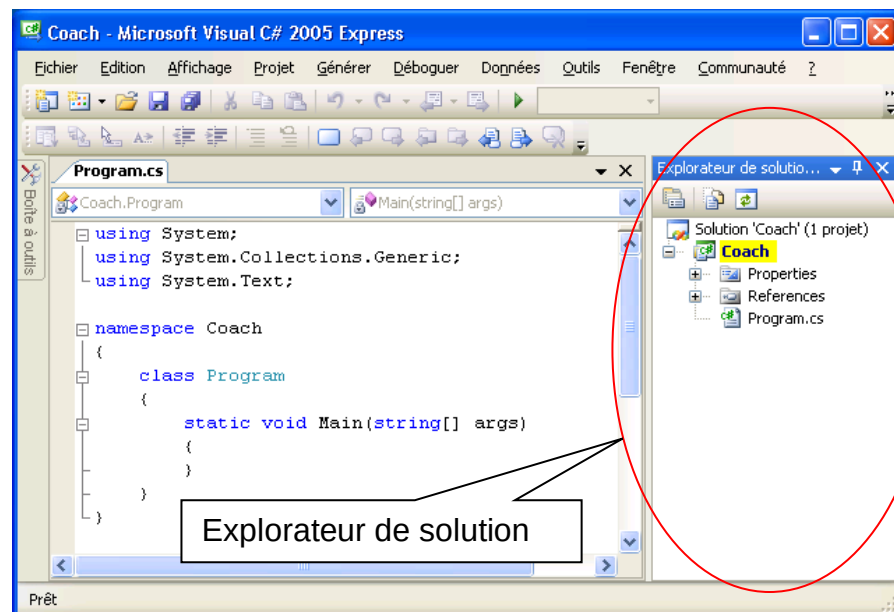


- Cliquez le bouton **OK** ;

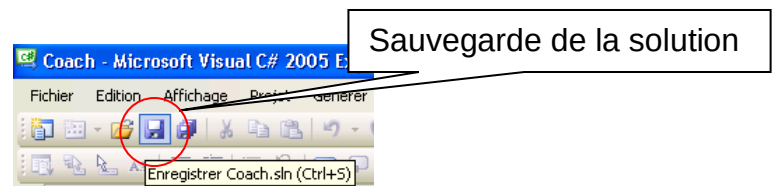
- Dans Visual C# Express, cliquez le menu **Affichage > Explorateur de solution** ;



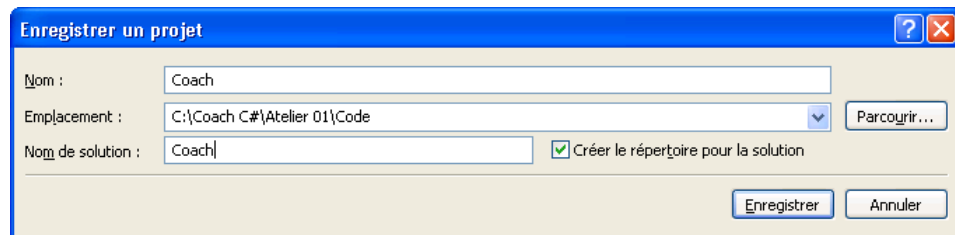
- L'explorateur de solution s'affiche sur la surface de travail de Visual C# Express,    droite si la fen  tre n'a pas   t   d  plac  e pr  alablement (vous pouvez positionner cette fen  tre o   vous voulez sur la surface de travail par simple glisser d  placer).



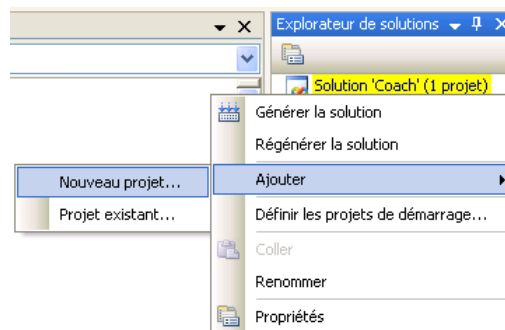
- Dans la barre d'outils, cliquez la disquette pour sauvegarder votre solution ;



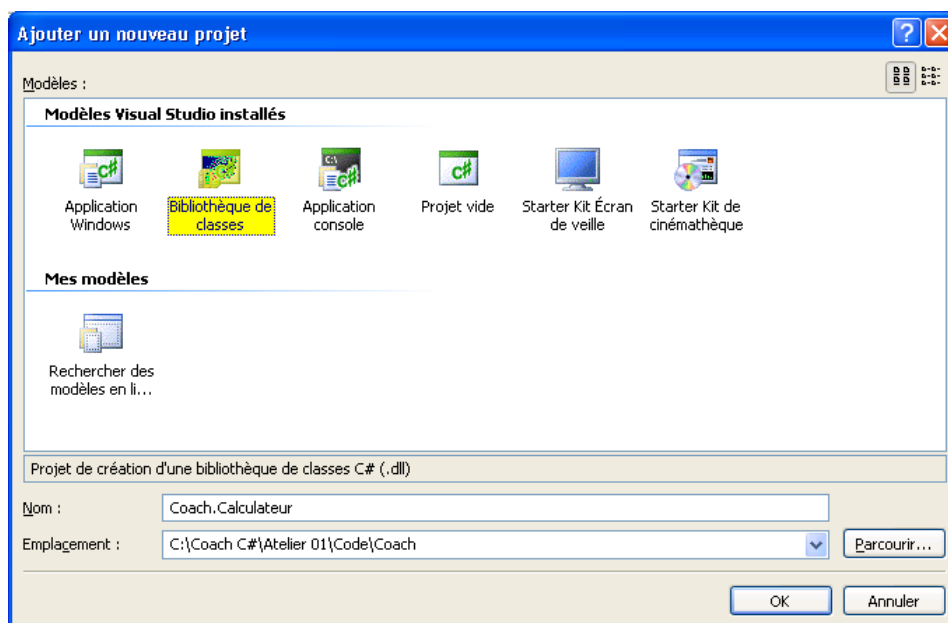
- Dans la boîte de dialogue **Enregistrer un projet**, indiquez votre répertoire de travail (par exemple **C:\Coach\Atelier 01\Code**), en cochant la case à cocher **Créer le répertoire pour la solution** ;



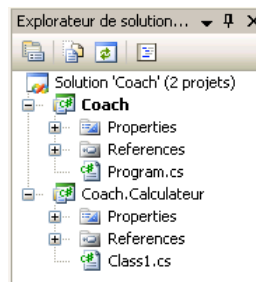
- Cliquez sur **Enregistrer** ;
- Dans l'explorateur de solution, faites un **clic-droit** sur la solution **Coach** et sélectionnez le menu **Ajouter > Nouveau projet ...**



- La boîte de dialogue **Ajouter un nouveau Projet** s'affiche. Dans cette boîte de dialogue, sélectionnez le modèle de projet **Bibliothèque de classes** et indiquez un nom de projet **Coach.Calculateur** ;



- Cliquez le bouton **OK** ; L'explorateur de solutions affiche maintenant deux projets : **Coach** et **Coach.Calculateur** ;



- Sauvegardez votre solution contenant maintenant les deux projets, en cliquant  ;



Visual C# 2005 Express est fourni avec les modèles de projet suivants :

- **Application Windows** : Ce modèle est utilisé pour développer des applications avec une interface utilisateur Windows à base de formulaire ;
- **Bibliothèque de classes** : Il s'agit ici de développer une bibliothèque de classes qui sera compilée sous forme d'un assemblage d'extension .dll ;
- **Application Console** : Ce modèle aide à la création d'une application en ligne de commande, très utilisé pour tester des services ou des librairies de classes ;
- **Projet vide** : Ce modèle sert de base à la création de tout type de programme, les différents éléments et paramètres devant être configurés manuellement ;
- **Starter Kit Ecran de veille, Starter Kit de cinémathèque** : ces deux modèles sont des exemples d'applications.



La liste des modèles disponibles en fonction des versions est sur le lien :

[http://msdn2.microsoft.com/fr-fr/library/ms236405\(vs.80\).aspx](http://msdn2.microsoft.com/fr-fr/library/ms236405(vs.80).aspx)



Ce n'est pas parce qu'un modèle n'est pas disponible dans la liste que vous ne pourrez pas développer le type d'application voulu. Il suffit de faire un projet vide, de rajouter manuellement toutes les références nécessaires, et de développer chacune des classes à partir d'une classe vide. Autrement dit, c'est faisable et complètement compilable, mais quelle perte de temps !



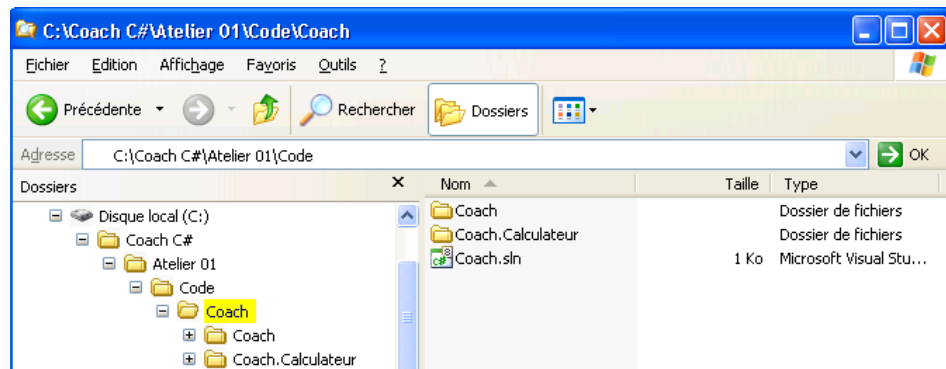
Vous pouvez développer vos propres modèles de projet, en regroupant ce que vous utilisez systématiquement dans vos développements :

[http://msdn2.microsoft.com/fr-fr/library/s365byhx\(VS.80\).aspx](http://msdn2.microsoft.com/fr-fr/library/s365byhx(VS.80).aspx)

3. Regardez ce qui a été créé sur le disque :

- Avec l'Explorateur Windows, regardez ce qu'il y a dans votre répertoire de travail (par exemple **C:\Coach C#\Atelier 01\Code**) ;

- Vous voyez un sous répertoire **Coach**, contenant un fichier **Coach.sln**, et deux sous répertoires **Coach** et **Coach.Calculateur** contenant respectivement vos deux projets.

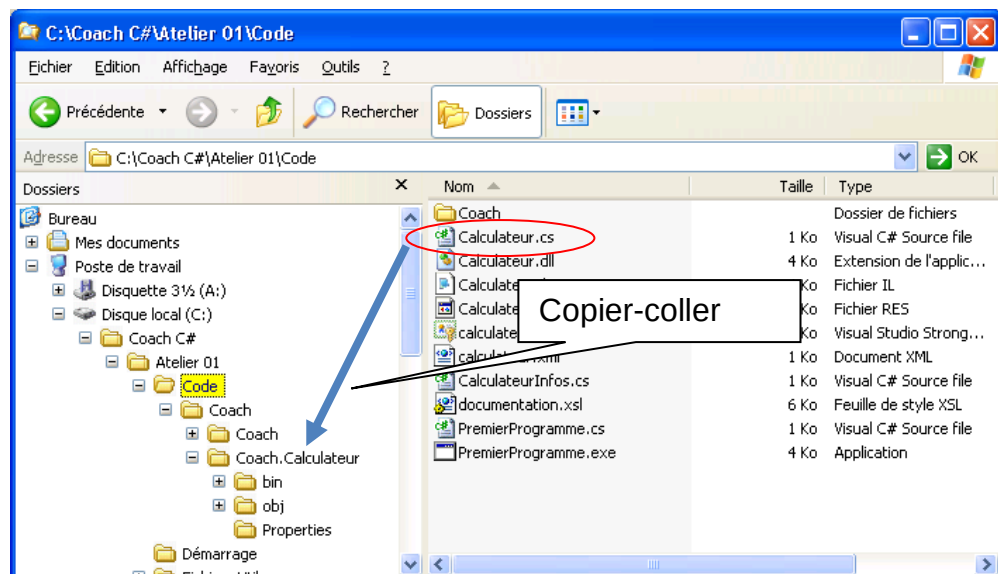


Le fichier **Coach.sln** est le fichier de solution. Vous pouvez l'ouvrir avec le bloc-notes. Il contient la définition de tous les projets composant la solution.

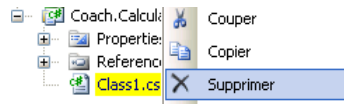


Dans chacun des sous-répertoires de projet, vous voyez un fichier d'extension **.csproj**. Il s'agit de votre fichier de projet, qui est utilisé par le compilateur (et MSBuild.exe) pour générer l'assemblage. Vous pouvez l'ouvrir avec le Bloc-notes, il est au format XML.

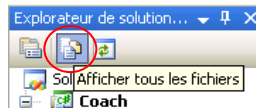
4. Ajoutez le code réalisé au premier exercice au projet **Coach.Calculateur** :
 - Avec l'explorateur Windows, à partir de votre répertoire de travail (par exemple **C:\Coach C#\Atelier 01\Code**) faites un **copier-coller** du fichier **Calculateur.cs** dans le répertoire du projet **Coach.Calculateur** (**C:\Coach C#\Atelier 01\Code\Coach\Coach.Calculateur**);



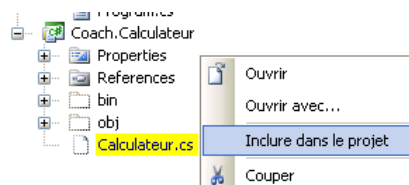
- Dans l'explorateur de solution de **Visual C# Express**, faites un clic-droit sur le fichier **Class1.cs** et sélectionnez le menu **Supprimer** ;



- Cliquez le bouton **OK** de la **boîte** de dialogue de confirmation de la suppression ;
- Sur la barre d'outil de l'Explorateur de Solutions, cliquez le bouton  pour afficher tous les fichiers ;



- Dans l'Explorateur de Solutions, sur le fichier **Calculateur.cs** (qui apparaît avec un icône grisé indiquant qu'il est extérieur au projet), faites un **clic-droit** et sélectionnez le menu **Inclure dans le projet** ;

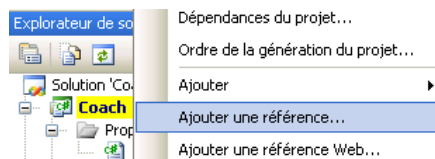


Le fichier apparaît maintenant avec un icône  indiquant qu'il fait maintenant parti du projet ;

- Dans la barre d'outils de Visual C# Express, cliquez le bouton  pour sauvegarder toutes les modifications réalisées ;

5. Modifiez le projet **Coach** pour utiliser la librairie de calcul :

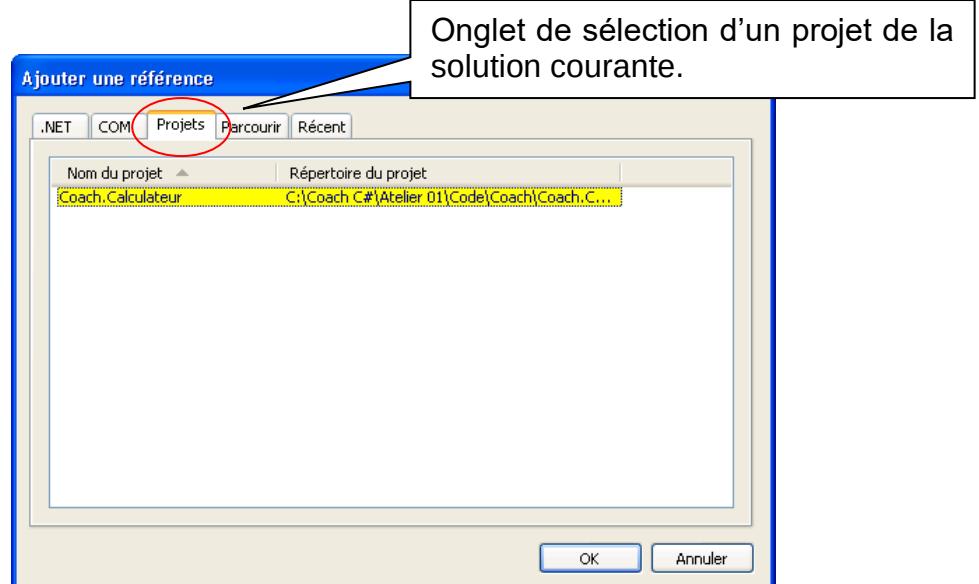
- Indiquez que le projet **Coach** va utiliser le calculateur de la librairie **Coach.Calculateur**, en faisant un **clic-droit** sur le projet **Coach** et en sélectionnant le menu **Ajouter une référence...** ;



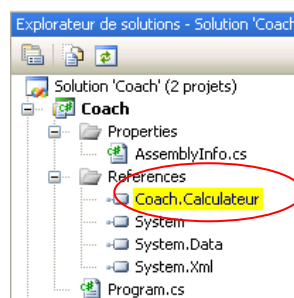
- Dans la boîte de dialogue **Ajouter une référence**, cliquez sur l'onglet **Projets**, et sélectionnez l'unique autre projet de la solution **Coach.Calculateur** ;



L'ajout d'une référence indique à l'IDE les noms d'objets à prendre en compte pour la vérification des noms lors du codage, et indique au compilateur les librairies à référencer et à copier.

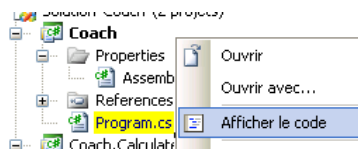


- Cliquez le bouton **OK** ;
- Regardez dans les références du projet **Coach**, une entrée **Coach.Calculateur** est ajoutée ;



Les références déjà présentes ont été ajoutées par le modèle de projet **Application Console** référencé lors de la création du projet **Coach**.

- Dans l'explorateur de solutions, sélectionnez le fichier **Program.cs** du projet **Coach** ;
- Faites un **clic droit** et sélectionnez le menu **Afficher le code** ; L'éditeur de code Visual C# Express s'affiche sur la surface de travail ;



Le code aurait aussi pu être affiché :

- en cliquant le bouton  (**Afficher le code**) de la barre d'outils de l'Explorateur de Solutions,
- ou en double-cliquant sur le fichier **Program.cs** dans l'Explorateur de Solutions,
- ou en sélectionnant le fichier **Program.cs** et en cliquant sur le **menu affichage > Code** de Visual C# Express,
- ou encore en sélectionnant le fichier **Program.cs** et en tapant la

touche **F7** ;

- Modifiez le code de **Program.cs** de façon à utiliser le calculateur ;

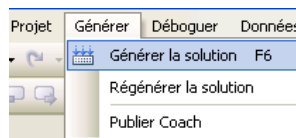
Code C#

```
using System;
using System.Collections.Generic;
using System.Text;

namespace Coach
{
    class Program
    {
        static void Main(string[] args)
        {
            System.Console.
                WriteLine("L'addition de 10 et 5 est : {0}.",
                    Calculateur.Ajouter(10, 5));

            System.Console.ReadKey();
        }
    }
}
```

- Dans la barre d'outil de Visual C# Express, cliquez le bouton  pour sauvegarder toutes les modifications réalisées ;
- Dans Visual C# Express, cliquez le menu **Générer > Générer la solution**, pour compiler les deux projets ;



La génération doit bien se passer. L'IDE vous l'indique en affichant le message suivant dans la barre de statut :



- Dans Visual C# Express, cliquez le menu **Déboguer > Démarrer le débogage**, pour lancer la solution ; une invite de commande s'affiche ;



- Appuyer sur une touche pour arrêter l'exécution du programme ;



Le code aurait aussi pu être généré en utilisant la touche **F6**. L'application aurait pu aussi être lancée en tapant la touche **F5**, ou en cliquant le bouton  dans la barre d'outils de Visual C# Express.



Les projets compilés sont sous le répertoire **bin\debug** de chaque projet respectif.



Sous chaque répertoire de compilation **bin\debug**, le programme **[nom de projet].vshost.exe** (par exemple **Coach.vshost.exe**) est le programme qui est utilisé par l'IDE en mode de débogage.



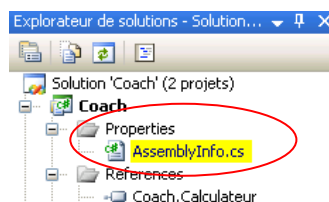
Vous pouvez agrandir la surface de travail de l'IDE en masquant les différentes fenêtres annexes (comme l'Explorateur de solutions). Pour ce faire, cliquez sur le bouton  de la fenêtre que vous voulez masquer. La fenêtre apparaît alors sous la forme d'un onglet qu'il suffit de survoler avec le curseur de la souris pour l'ouvrir. Pour fixer une fenêtre masquée, cliquez sur le bouton  de cette fenêtre.



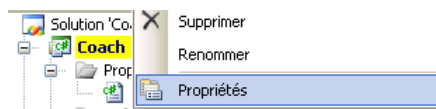
Si vous avez perdu une fenêtre (oui, cela arrive parfois ☺), pensez au menu **Affichage** (et aussi **Affichage > Autres fenêtres** et **Affichage > Barres d'outils**) de Visual C# Express, qui contient la quasi-totalité des fenêtres accessibles avec votre version de Visual Studio. Néanmoins, d'autres fenêtres peuvent parfois être ouvertes avec des menus fortement en relation avec le traitement de la fenêtre. C'est par exemple le cas du menu **Données > Afficher les sources de données**.



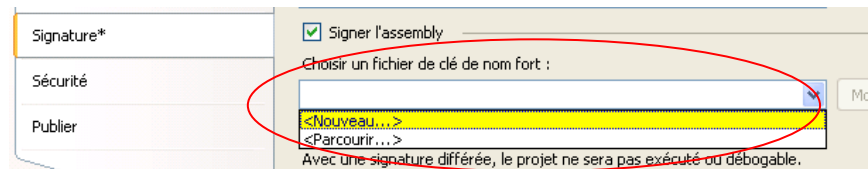
Sous chaque projet, dans le sous répertoire **Properties**, vous avez un fichier **AssemblyInfo.cs** qui contient les informations relatives à l'assemblage comme les numéros de version ou le copyright.



En cliquant droit sur le nom d'un projet et en sélectionnant le menu **Propriétés**, l'ensemble des propriétés du projet en cours sont affichées.



Parmi ces propriétés, vous retrouverez sur l'onglet **Signature** celle qui permet de signer avec un nom fort l'assemblage. Cela vous évite d'avoir à générer avec l'utilitaire **sn.exe** le fichier de paire de clés, et de l'associer en ligne de commande lors de la compilation.



Vous pouvez entrer le même nom que celui de votre projet pour le fichier de clé. Il n'est pas nécessaire de mettre un mot de passe.

3.2 Apprivoiser l'environnement de développement

Déroulement de l'exercice :

1. Utilisez l'IntelliSense :

- Dans l'Explorateur de Solutions, sélectionnez le fichier **Program.cs** du projet **Coach** ;
- Faites un **clic droit** et sélectionnez le menu **Afficher le code** ; L'éditeur de code Visual C# Express s'affiche sur la surface de travail ;
- A la fin de la ligne **System.Console.ReadKey()** ; , tapez la touche **Entrée** pour ajouter une nouvelle ligne de programme ;
- Commencez à taper le mot **Sys** ;
- Une fenêtre proposant les différents objets est ouverte sous le curseur, en surlignant le mot le plus proche du début que vous venez de taper. C'est l'IntelliSense ;
- Tapez un **.** (**point**) ;
- Le contenu de **System** s'affiche ;
- Continuer à taper **cons** ;
- L'IntelliSense vous propose automatiquement le mot Console ;

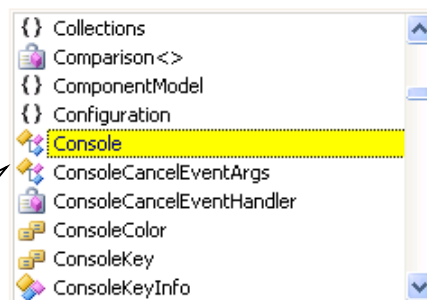
```
namespace Coach
{
    class Program
    {
        static
        {
            System.Console.
            WriteLine("L'addition de
            Calculateur.Ajouter()

            System.Console.ReadKey();
            System.con
        }
    }
}
```

En vert apparaissent les lignes modifiées et déjà sauvegardées, et en jaunes les lignes modifiées mais pas encore sauvegardées.

IntelliSense affiche une aide rapide de l'élément sélectionné

IntelliSense propose les objets accessibles dans le contexte en cours.



class System.Console
Représente les flux d'entrée, de sortie héritée.

- Fermez **Program.cs** sans le sauvegarder ;



IntelliSense est un mécanisme automatique d'auto-sélection qui intervient comme une aide à la programmation et un accès à la documentation. Il propose sans ambiguïté les noms des types, des méthodes, des fonctions et des objets accessibles dans le contexte, et aussi des aides au codage sous forme de blocs de codes préprogrammés.



Les touches à connaître avec IntelliSense sont :

- Le **point (.)**, pour afficher les membres d'un type ;
- Le **Tab (Tabulation)**, pour sélectionner l'élément surligné dans IntelliSense ;
- Le **Ctrl Espace** pour vérifier l'écriture d'un terme présent dans l'IntelliSense, et en finir la tape si le mot (ou le début de mot) tapé n'a pas d'ambiguïté ;
- Le **Tab Tab (double tabulation)** pour insérer un extrait de code.



Lors de la programmation, si IntelliSense ne reconnaît pas un terme alors que vous vous attendiez à ce qu'il soit reconnu, c'est que :

- Ou vous êtes positionné au mauvais endroit ;
- Ou la portée de l'objet le rend inaccessible dans votre contexte ;
- Ou vous avez oublié de référencer la librairie fournissant cet objet dans votre projet.

Dans tous les cas, si IntelliSense ne vous propose pas le terme attendu, vous obtiendrez une erreur de compilation.

2. Utilisez les extraits de code :

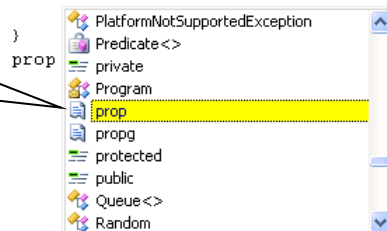


Un extrait de code est un bloc préprogrammé reprenant les structures de base de C# pour vous aider à aller plus vite dans vos développements.

- Dans l'explorateur de solutions, sélectionnez le fichier **Program.cs** du projet **Coach** ;
- Faites un **clic droit** et sélectionnez le menu **Afficher le code** ; L'éditeur de code Visual C# Express s'affiche sur la surface de travail ;
- **Ajoutez** une ligne vierge après l'accolade fermante **}** de la **méthode static void Main** ;
- Tapez le début de mot **prop** ; IntelliSense s'affiche ;

```
class Program
{
    static void Main(string[] args)
    {
        System.Console.
            WriteLine("L'addition de 10 et 5 est : {0}.",
                Calculateur.Ajouter(10, 5));
    }
}
```

```
System.Console.ReadKey();
```



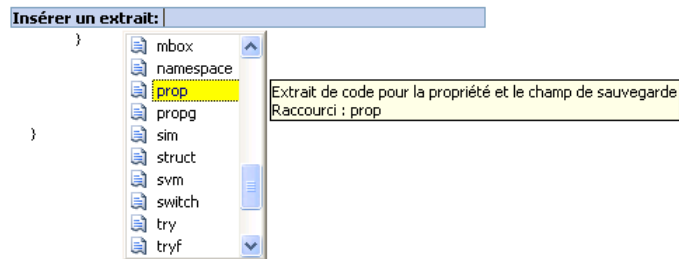
Le logo  indique les extraits de code.

prop
Extrait de code pour la propriété et le champ de sauvegarde

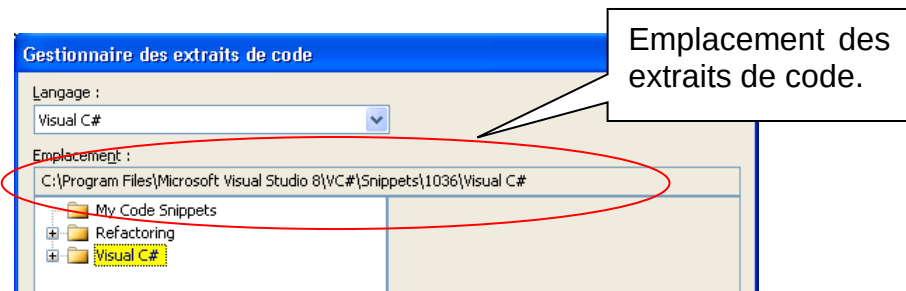
- Tapez **Tab Tab** (double tabulation) pour insérer un extrait de code dans le programme ;
- Fermez **Program.cs** sans le sauvegarder ;



Pour ajouter un extrait de code, vous pouvez aussi vous positionner à l'endroit dans l'éditeur de code où vous souhaitez l'insérer, faire un clic-droit et sélectionner le menu **Insérer un extrait ...**
Une fenêtre d'aide à l'insertion apparaît :



Pour visualiser et gérer l'ensemble des extraits de code, sélectionnez le menu **Outils > Gestionnaire des extraits de code...** de Visual C# Express. Cette fenêtre vous indique notamment l'endroit où ces extraits de code sont installés (le répertoire 1036 indique le langage français).



Visual C# Express arrive avec relativement peu d'extraits de code (c'est aussi vrai avec les autres versions de Visual Studio C#). Les développeurs C# doivent être considérés comme des pros de la ligne de code ... ☺

Néanmoins, si vous êtes comme moi et que vous avez horreur de taper deux fois la même ligne (surtout si elle a déjà été écrite), vous pouvez charger un ensemble d'extraits de code à l'adresse suivante :

<http://msdn2.microsoft.com/fr-fr/vstudio/aa718338.aspx>

Dans ce cas, la procédure à suivre pour les installer est la suivante :

- Chargez les extraits de code (code snippets) à partir de l'adresse ci-dessus, qui vous charge le fichier **Code_Snippets.msi** ;
- Lancez l'installation des extraits de code en double-cliquant ce fichier **Code_Snippets.msi** ;
- Passez les étapes d'installation, et à l'étape **Select Installation Folder**, indiquez le chemin d'installation des extraits de code sur votre machine (cf. ci-dessus l'emplacement indiqué dans le gestionnaire des extraits), certainement **C:\Program**

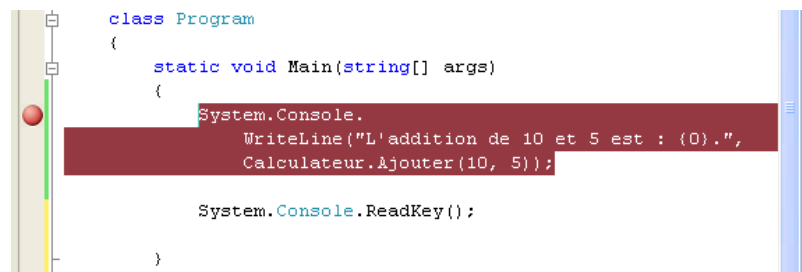
Files\Microsoft Visual Studio 8\VC#\Snippets\1036\Visual C#

- Finissez l'installation en laissant les autres étapes inchangées.

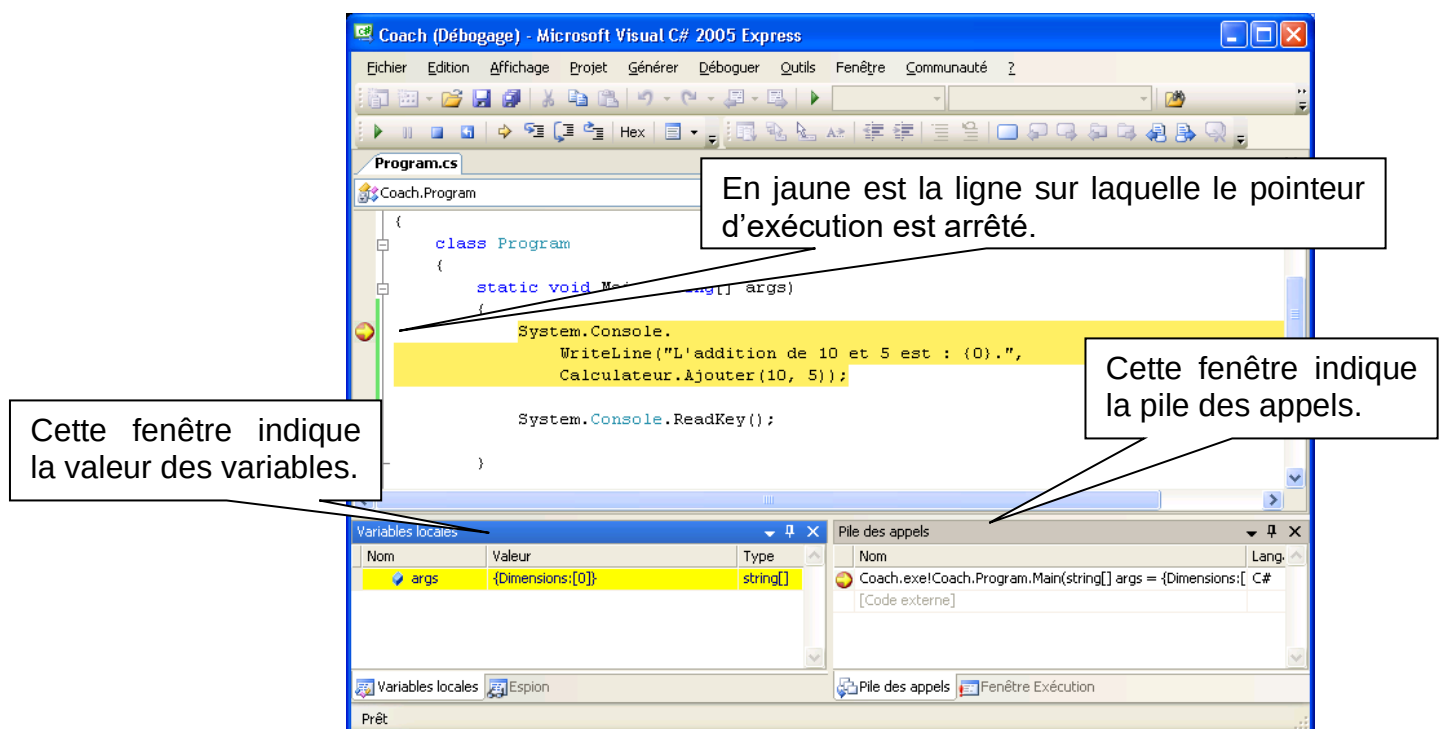
Une fois cette procédure réalisée, vérifiez la présence de vos extraits de code dans le gestionnaire des extraits de code, en faisant un clic droit dans le code et en sélectionnant le menu **Insérer un extrait ...** Il y a de nombreux extraits pour traiter les aspects les plus classiques de la programmation. Remarque : les noms sont en anglais pour le moment.

3. Déboguez votre application :

- Dans l'Explorateur de Solutions, sélectionnez le fichier **Program.cs** du projet **Coach** ;
- Faites un **clic droit** et sélectionnez le menu **Afficher le code** ; L'éditeur de code Visual C# Express s'affiche sur la surface de travail ;
- Faites un clic dans la marge de l'éditeur de code ; Un point rouge apparaît. Vous venez d'insérer un point d'arrêt :



- Lancez l'application en tapant la touche **F5** ; l'exécution s'arrête sur la ligne du point d'arrêt, et vous pouvez entrer en débogage de votre code. Visual C# Express propose de nombreux outils d'aide.



- Arrêtez l'exécution et fermez tous les programmes ;



Pour tout savoir sur le débogage avec Visual Studio :

[http://msdn2.microsoft.com/fr-fr/library/sc65sadd\(VS.80\).aspx](http://msdn2.microsoft.com/fr-fr/library/sc65sadd(VS.80).aspx)

4 Enrichir le comportement de l'application

Dans cet exercice, vous allez apprendre à :

- Ajouter différents types de bloc de code à votre application ;
- Manipuler les paramètres passés à une méthode ;
- Trouver et utiliser quelques objets du Framework.

Objectif

L'objectif de cet exercice est de bien différencier ce qui est fourni par le langage C# de ce qui est issu du Framework 4.0.

Contexte fonctionnel

Vous allez reprendre la fonction d'addition du calculateur, en ajoutant la possibilité d'ajouter n chiffres. Ensuite nous structurerons le code pour le protéger (succinctement) des erreurs, et utiliserons quelques objets du Framework.

4.1 Modifier la fonction d'ajout



En fait, une fonction d'ajout utilise rarement un nombre fini de paramètres, et de plus est de type entier. L'idée est ici de modifier la méthode afin de pouvoir ajouter un nombre n de valeurs non entières.

Déroulement de l'exercice :

1. Modifiez la méthode `Calculateur.Ajouter` :

- Dans l'Explorateur de Solutions, sélectionnez le fichier **Calculateur.cs** du projet **Coach.Calculateur** ;
- Faites un **clic droit** et sélectionnez le menu **Afficher le code** ; L'éditeur de code Visual C# Express s'affiche sur la surface de travail ;
- Modifiez la déclaration de la méthode **Ajouter** de façon à ce qu'elle accepte un tableau de réels (double), et renvoie aussi une valeur réelle ;

Code C#

```
public static int double Ajouter(double[] Valeurs int Valeur01, int Valeur02)
```

Le type de renvoi précède le nom de la méthode

Le **s** est une convention de nommage indiquant plusieurs valeurs

Le type précède le nom de la variable

Les crochets indiquent un tableau de réels

(Ne pas oublier de modifier les commentaires de documentation pour refléter les modifications)

- Ajoutez quelques lignes vides avant le mot clé **return** de cette méthode ;

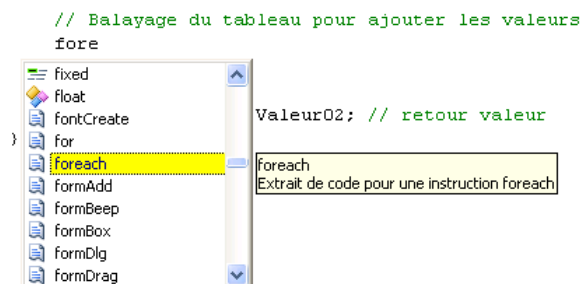
- Ajouter la définition d'une variable **result** de type réel en début de méthode (n'oubliez pas de commenter votre code ☺);

Code C#

```
public static double Ajouter(double[] Valeurs)
{
    // Definition des variables locales
    double result;
```

Le type précède le nom de la variable

- Commencez à taper le mot **fore** : l'IntelliSense vous propose l'extrait de code **foreach** ;



- Tapez **Tab Tab** (double tabulations) et l'extrait de code est inséré ;

```
// Balayage du tableau pour ajouter les valeurs
foreach (object var in collection_to_loop)
{
}
```



L'instruction de boucle **foreach** navigue dans tous les éléments d'une liste ou d'un tableau, en renvoyant à chaque itération l'élément en cours. Il s'arrête automatiquement en fin de liste. Pour sortir de la boucle avant la fin, utilisez l'ordre **break** ;

- Modifier le code de façon à ajouter au résultat la valeur de l'élément courant à chaque itération ;

Code C#

```
// Balayage du tableau pour ajouter les valeurs
foreach (double valeur in Valeurs)
{
    result = result + valeur;
}
```

Le type de l'élément courant

Le nom de l'élément courant

La liste contenant les éléments à balayer

- Modifiez le retour de la méthode afin de renvoyer la variable **result**.

Code C#

```
return result ; // retour de la valeur
```

Le code complet de la méthode d'ajout est le suivant :

Code C#

```

/// <summary>
/// Cette fonction ajoute n valeurs réelles
/// </summary>
/// <param name="Valeurs">Tableau de valeurs à ajouter</param>
/// <returns>le réel en résultat</returns>
public static double Ajouter(double[] Valeurs)
{
    // Definition des variables locales
    double result;

    // Balayage du tableau pour ajouter les valeurs
    foreach (double valeur in Valeurs)
    {
        result = result + valeur;
    }

    return result ; // retour de la valeur
}

```

- Sauvegardez votre travail en cliquant sur le bouton  de la barre d'outils, et lancez une compilation en tapant la touche **F6** ;
- Vous devez obtenir (au moins ...) un message d'erreur ;

Liste d'erreurs					
1 erreur		0 avertissements		0 messages	
	Description	Fichier	Ligne	Colonne	Projet
1	Utilisation d'une variable locale non assignée 'result'	Calculateur.cs	32	26	Coach.Calculateur



Si la liste des erreurs n'est pas visible dans Visual C# Express, cliquez le menu **Affichage > Liste d'erreurs**, ou cliquez les touches **Ctrl+WE** ;



A la différence d'autres langages (comme VB.Net par exemple), Visual C# nécessite l'assignation systématique des variables à une valeur, c'est-à-dire qu'il ne décide pas à la place du développeur quel est la valeur par défaut des variables. Cela évite des bogues souvent difficile à trouver !



Dans la boucle, une manière plus élégante d'exprimer le cumul du résultat avec l'élément courant est d'utiliser un opérateur de calcul d'ajout et d'assignation **+=**

La ligne devient ainsi **result += valeur** ; Il existe l'équivalent pour la soustraction et la multiplication **-=** et ***=** respectivement.



C# propose un large éventail d'opérateurs. Ces derniers sont des symboles qui spécifient les opérations à effectuer dans une expression. C# prédéfinit les opérateurs arithmétiques et logiques habituels, ainsi que de nombreux autres, comme illustré à l'adresse suivante :

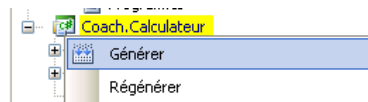
[http://msdn2.microsoft.com/fr-fr/library/6a71f45d\(VS.80\).aspx](http://msdn2.microsoft.com/fr-fr/library/6a71f45d(VS.80).aspx)

- Modifiez le code afin d'assigner la **valeur par défaut 0** à **result** ; vous pouvez en profiter pour modifier le cumul du résultat ;

Code C#

```
// Definition des variables locales
double result = 0;
```

- Sauvegardez votre travail en cliquant sur le bouton  de la barre d'outils ;
- Dans l'Explorateur de Solutions, faites un **clic-droit** sur le projet **Coach.Calculateur** et cliquez le menu **Générer** ;



La génération de la nouvelle librairie de calcul.

- Lancez une compilation de toute votre solution en tapant la touche **F5** ;
- Vous devez obtenir un message d'erreur ;

Liste d'erreurs					
1 erreur 0 avertissements 0 messages					
	Description	Fichier	Ligne	Colonne	Projet
1	Aucune surcharge pour la méthode 'Ajouter' ne prend d'arguments '2'	Program.cs	13	17	Coach



En double-cliquant sur la ligne d'erreur, l'éditeur de code vous positionne sur la ligne de votre programme qui pose problème.



L'erreur vient du fait que la méthode attend une variable (tableau de réel), alors que l'appel du calculateur passe deux paramètres.



Pour corriger cette erreur, le mot clé **params** vous permet de spécifier un paramètre de méthode qui accepte un argument avec un nombre d'arguments variable. Aucun paramètre supplémentaire n'est autorisé après le mot clé **params** dans une déclaration de méthode et un seul mot clé **params** est autorisé dans une telle déclaration.

- Modifiez le code afin de préciser le mot clé **params** sur l'appel de méthode ;

Code C#

```
public static double Ajouter(params double[] Valeurs)
```

- Sauvegardez votre travail en cliquant sur le bouton  de la barre d'outils ;
- Lancez une compilation de toute votre solution en tapant la touche **F6** ; Tout se compile maintenant correctement.

Le code complet de la méthode **Calculateur.Ajouter** est le suivant :

Code C#

```
/// <summary>
/// Cette fonction ajoute n valeurs réelles
/// </summary>
/// <param name="Valeurs">Tableau de valeurs à ajouter</param>
/// <returns>le réel en résultat</returns>
public static double Ajouter(params double[] Valeurs)
{
    // Definition des variables locales
    double result = 0;

    // Balayage du tableau pour ajouter les valeurs
    foreach (double valeur in Valeurs)
    {
        result += valeur;
    }

    return result ; // retour de la valeur
}
```



Dans cet exercice, nous avons utilisé une boucle de type **foreach**. Vous pouvez créer des boucles avec différentes instructions d'itération. Les instructions d'itération font exécuter des instructions incorporées un certain nombre de fois, selon les critères de terminaison des boucles. Ces instructions sont exécutées généralement dans l'ordre où elles sont écrites. Pour tout savoir sur les instructions de boucle :

[http://msdn2.microsoft.com/fr-fr/library/32dbftby\(VS.80\).aspx](http://msdn2.microsoft.com/fr-fr/library/32dbftby(VS.80).aspx)

2. Modifiez la méthode Calculateur.Ajouter :

- Dans l'Explorateur de Solutions, sélectionnez le fichier **Program.cs** du projet **Coach** ;
- Faites un **clic droit** et sélectionnez le menu **Afficher le code** ; L'éditeur de code Visual C# Express s'affiche sur la surface de travail ;



Vous voyez que la méthode **Main**, qui est le point d'entrée de l'application, a un paramètre **args** qui est un tableau de chaînes de caractères : **static void Main(string[] args)**

L'idée est de modifier l'application de façon à ajouter les valeurs passées en argument sur la méthode **Main**.



Pour implémenter cette nouvelle fonction, il va falloir transformer le type des arguments fournis en entrée (chaîne de caractères) en réels, qui sont acceptés par la méthode **Calculateur.Ajouter**. Cette conversion de types est appelée **Casting**.



Plus d'informations et des liens utiles concernant le casting et les opérateurs de cast sont à l'URL suivante :

[http://msdn2.microsoft.com/fr-fr/library/ms173105\(VS.80\).aspx](http://msdn2.microsoft.com/fr-fr/library/ms173105(VS.80).aspx)

- Dans le **Main**, définissez et assignez deux variables **valeursAAfficher** et **valeursACalculer** ;

Code C#

```
// Définition et assignation des variables locales
string valeursAAfficher = string.Empty;
double[] valeursACalculer = new double[args.Length];
```

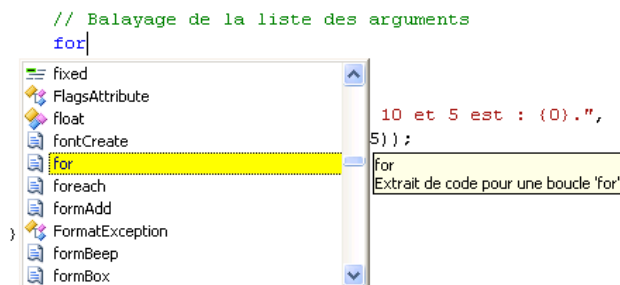
Assignation à une chaîne vide

Assignation à un nouveau tableau de réels, comportant autant d'éléments que le nombre d'arguments passés.



Dans cette exercice nous allons balayer le tableau d'arguments avec une boucle de type `for (int i = 0; i < Length; i++)`

- Commencez à taper le mot **for** : l'IntelliSense vous propose l'extrait de code **for** ;



- Tapez **Tab Tab** (double tabulations) et l'extrait de code est inséré ;

Valeur de départ

Valeur de fin

```
// Balayage de la liste des arguments
for (int i = 0; i < length; i++)
{
    // ...
}
```

Incrément

- Indiquez que la valeur de fin est le nombre d'éléments passés en argument, c'est-à-dire **args.Length** ;
- A l'intérieur du bloc **for**, ajoutez les deux lignes suivantes :

Code C#

```
// Balayage de la liste des arguments
for (int i = 0; i < args.Length; i++)
{
    // Chaîne pour l'affichage
    valeursAAfficher += args[i] + " ";
    // Tableau pour le calcul
    valeursACalculer[i] = Convert.ToDouble(args[i]);
}
```

Une concaténation de base est réalisée



La classe **Convert** convertit un type de données de base en un autre type de données de base. Plus d'informations :

[http://msdn2.microsoft.com/fr-fr/library/system.convert\(VS.80\).aspx](http://msdn2.microsoft.com/fr-fr/library/system.convert(VS.80).aspx)

- Modifiez l'appel du calculateur de façon à référencer maintenant nos deux variables ;

Code C#

```
System.Console.  
    WriteLine("L'addition de {0} est : {1}.",  
        valeursAAfficher,  
        Calculateur.Ajouter(valeursACalculer));
```

- Sauvegardez votre travail en cliquant sur le bouton  de la barre d'outils ;
- Lancez une compilation de toute votre solution en tapant la touche **F6** ; Tout se compile correctement.

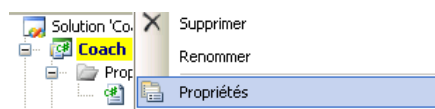
Le code complet de la méthode **Program.Main** est le suivant :

Code C#

```
static void Main(string[] args)  
{  
    // Définition et assignation des variables locales  
    string valeursAAfficher = string.Empty;  
    double[] valeursACalculer = new double[args.Length];  
  
    // Balayage de la liste des arguments  
    for (int i = 0; i < args.Length; i++)  
    {  
        // Chaine pour l'affichage  
        valeursAAfficher += args[i] + " ";  
        // Tableau pour le calcul  
        valeursACalculer[i] = Convert.ToDouble(args[i]);  
    }  
  
    System.Console.  
        WriteLine("L'addition de {0} est : {1}.",  
            valeursAAfficher,  
            Calculateur.Ajouter(valeursACalculer));  
    System.Console.ReadKey();  
}
```

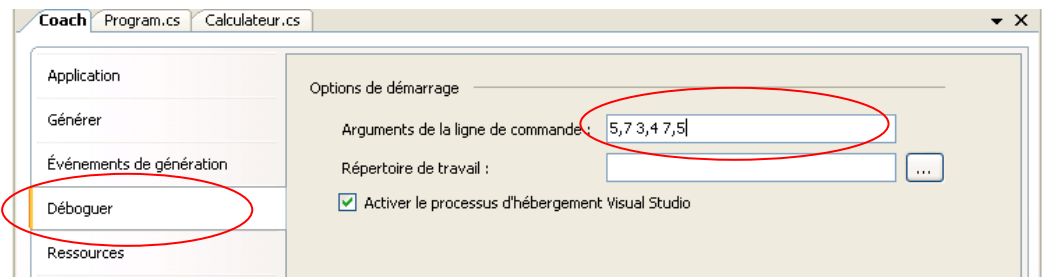
3. Testez votre programme en passant des paramètres :

- Faites un clic droit sur le projet **Coach** et en sélectionnez le menu **Propriétés** ;



- Sélectionnez l'onglet **Déboguer** ;
- Sur la propriété **Arguments de la ligne de commande**, indiquez quelques valeurs à ajouter. Attention, respectez bien la culture

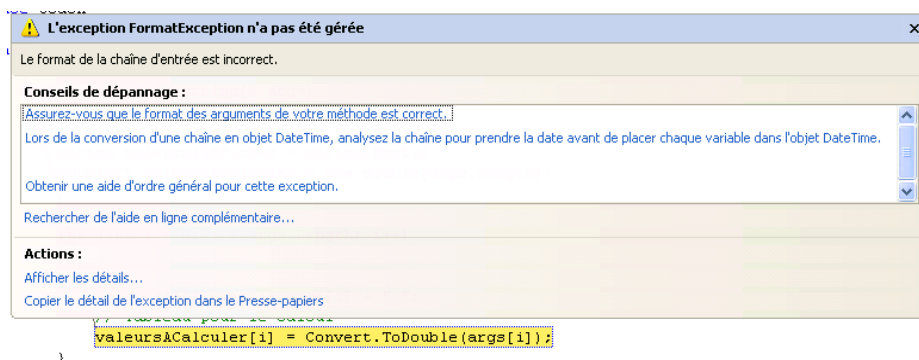
configurée sur votre poste, c'est-à-dire que si vous êtes en France, saisissez des réels avec des virgules (et non pas des points – par exemple 5,7 3,4 7,5) ;



- Dans la barre d'outil de Visual C# Express, cliquez le bouton  pour sauvegarder les modifications réalisées ;
- Cliquez maintenant le bouton  dans la barre d'outils de Visual C# Express, pour lancer l'application. Tout fonctionne correctement.

4. Améliorez la robustesse de votre code :

- Faites un clic droit sur le projet **Coach** et en sélectionnez le menu **Propriétés** ;
- Sélectionnez l'onglet **Déboguer** ;
- Sur la propriété **Arguments de la ligne de commande**, ajoutez à la liste existante une valeur non numérique en argument (par exemple **A**) ;
- Dans la barre d'outils de Visual C# Express, cliquez le bouton  pour sauvegarder toutes les modifications réalisées ;
- Cliquez maintenant le bouton  dans la barre d'outils de Visual C# Express, pour lancer l'application. Vous obtenez une erreur ;



Cette erreur vient du fait que l'argument contient une chaîne de caractères (**A**) qui n'est pas convertible en réel. Le code n'est donc pas robuste.



C# offre une assistance intégrée pour gérer des situations anormales, appelées exceptions, qui peuvent avoir lieu pendant l'exécution de votre programme. Ces exceptions sont traitées par du code situé hors du déroulement normal du contrôle. Dans notre cas, nous allons utiliser un bloc **try ... catch ... finally**. Le bloc **try** contient le code protégé risquant

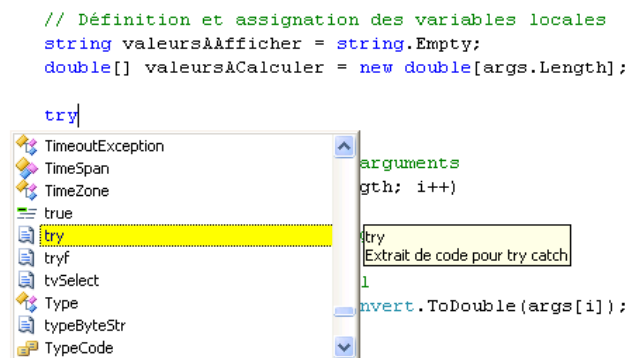
de provoquer l'exception. Le bloc est exécuté jusqu'à la levée d'une exception ou jusqu'à sa réussite totale. Il est suivi d'une ou de plusieurs clauses **catch**, qui désignent les gestionnaires des différentes exceptions. Tandis que **catch** est utilisé pour gérer les exceptions qui se produisent dans un bloc d'instructions, **finally** sert à garantir l'exécution d'un bloc d'instructions de code quelle que soit la méthode de sortie du bloc **try** précédent. Une utilisation de la combinaison de **catch** et **finally** permet d'obtenir et d'utiliser des ressources dans un bloc **try**, de traiter des circonstances exceptionnelles dans un bloc **catch** et de libérer les ressources dans le bloc **finally**.



Plus d'information sur les blocs de gestion des exceptions sont à l'url suivante :

[http://msdn2.microsoft.com/fr-fr/library/s7fekhdy\(VS.80\).aspx](http://msdn2.microsoft.com/fr-fr/library/s7fekhdy(VS.80).aspx)

- Dans la barre d'outil de Visual C# Express, cliquez le bouton  pour arrêter l'exécution du programme en mode de débogage ;
- Juste après les deux lignes de définition et d'assignation des variables locales, ajoutez deux lignes vides ;
- Commencez à taper le mot **try** : l'IntelliSense vous propose l'extrait de code **try** ;



- Tapez **Tab Tab (double tabulations)** et l'extrait de code est inséré ;
- Dans le bloc **catch**, retirez l'instruction **throw** ;
- A la fin du bloc **catch**, ajoutez un bloc **finally** ;

Code C#

```
try
{
}
catch (Exception)
{
}
finally
{
}
```

- Faites un **glisser-déplacer** du code de balayage des arguments et d'affichage du calcul à l'intérieur du bloc **try** ;
- Faites un **glisser-déplacer** du code de lecture d'une touche (ReadKey) dans le bloc **finally** ;
- Ajoutez le code d'affichage d'un message d'erreur sur la console dans le bloc **catch** ;

Code C#

```
static void Main(string[] args)
{
    // Définition et assignation des variables locales
    string valeursAAfficher = string.Empty;
    double[] valeursACalculer = new double[args.Length];

    try
    {
        // Balayage de la liste des arguments
        for (int i = 0; i < args.Length; i++)
        {
            // Chaîne pour l'affichage
            valeursAAfficher += args[i] + " ";
            // Tableau pour le calcul
            valeursACalculer[i] = Convert.ToDouble(args[i]);
        }

        System.Console.
            WriteLine("L'addition de {0} est : {1}.",
                valeursAAfficher,
                Calculateur.Ajouter(valeursACalculer));
    }
    catch (Exception)
    {
        System.Console.
            WriteLine("Les arguments doivent être numériques");
    }
    finally
    {
        System.Console.ReadKey();
    }
}
```

- Sauvegardez votre travail en cliquant sur le bouton  de la barre d'outils ;
- Lancez une compilation de toute votre solution en tapant la touche **F6** ; Tout se compile correctement.
- Cliquez maintenant le bouton  dans la barre d'outils de Visual C# Express, pour lancer l'application. La console vous indique maintenant le message d'erreur ;
- Faites un clic droit sur le projet **Coach** et en sélectionnez le menu **Propriétés** ;
- Sélectionnez l'onglet **Déboguer** ;
- Sur la propriété **Arguments de la ligne de commande**, retirez la valeur non numérique ;

- Dans la barre d'outils de Visual C# Express, cliquez le bouton  pour sauvegarder toutes les modifications réalisées ;
- Cliquez de nouveau le bouton  dans la barre d'outils de Visual C# Express, pour lancer l'application. La console vous indique maintenant le résultat du calcul ;



Des exemples et guides de pratiques complets concernant la gestion des exceptions sont disponibles à l'adresse suivante :

<http://msdn2.microsoft.com/fr-fr/library/ms954830.aspx>

4.2 Ajouter des objets sympathiques



Comme nous venons de le voir, Visual C# fournit uniquement une façon d'exprimer un besoin fonctionnel, en respectant la grammaire définie. Les types de base (string, double, int, etc.) sont fournis par le Framework 4.0, qui fournit aussi tous les autres objets dont nous pouvons avoir besoin lors de la programmation. Il y a aujourd'hui dans le Framework certainement plusieurs dizaines de milliers d'objets différents.

La conversion que nous avons réalisée précédemment a été faite au travers d'un objet **Convert**. Si vous voulez lire un port série, gérer des processus, compresser des fichiers, etc. vous devrez utiliser le bon objet du Framework. En fait, tout est objet avec .Net.



Dans cet exercice, nous allons indiquer le résultat de calcul dans une boîte de dialogue, et ajouter un petit icône dans la barre des tâches de Windows XP en bas à droite de l'écran.

Déroulement de l'exercice :

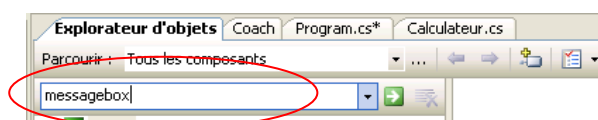
1. Ecrivez les messages dans une boîte de message :

- Dans Visual C# Express, sélectionnez le menu **Affichage > Autres fenêtres > explorateur d'objets** ;

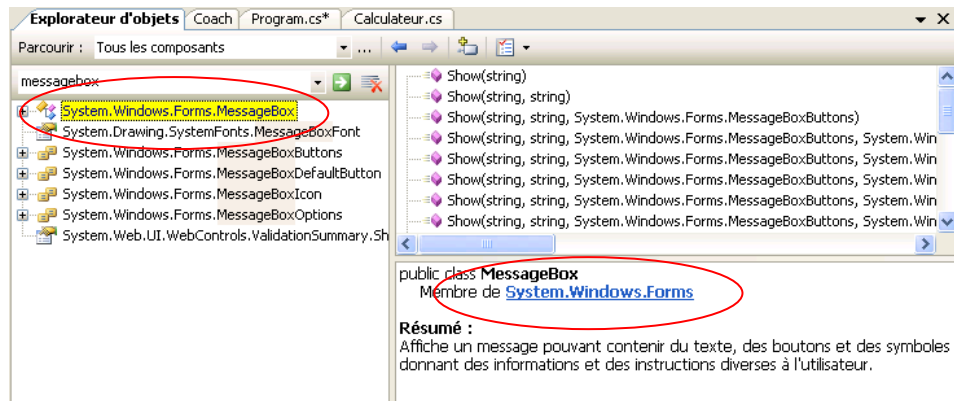


L'explorateur d'objets de Visual Studio affiche les métadonnées de tous les objets connus par l'environnement de développement. C'est un excellent outil pour localiser un objet particulier, quand vous ne savez plus où il se trouve.

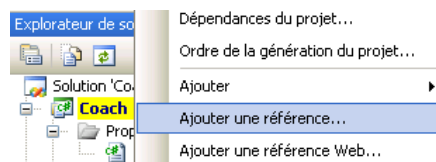
- Dans la fenêtre **<Rechercher>**, tapez **messagebox** ;



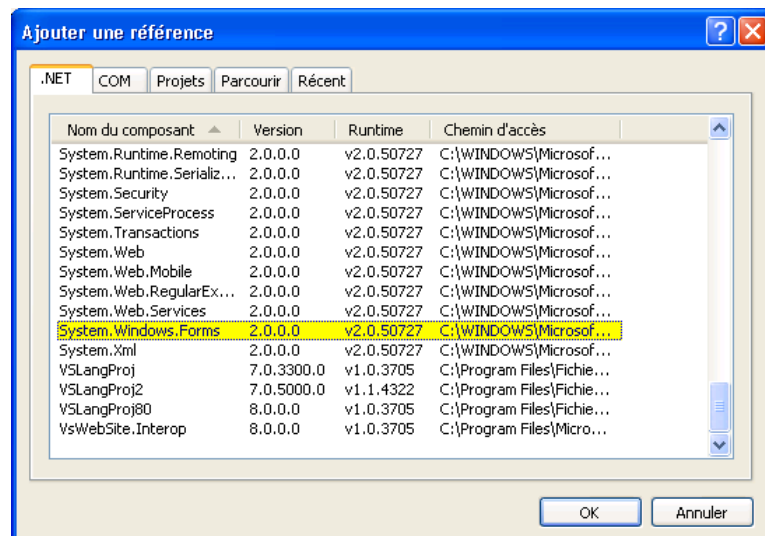
- Cliquez sur le bouton  pour lancer la recherche ;
- L'explorateur d'objets affiche tous les objets contenant la chaîne cherchée. Dans notre cas, nous voyons que la boîte de message est localisée dans la librairie **System.Windows.Forms** ;



- Indiquez que le projet **Coach** doit utiliser la librairie **System.Windows.Forms**, en faisant un **clic-droit** sur le projet **Coach** et en s  lectionnant le menu **Ajouter une r  f  rence...** ;



- Dans la **bo  te de dialogue Ajouter une r  f  rence**, cliquez sur l'onglet **.Net**, d  roulez la liste des composants et s  lectionnez la librairie **System.Windows.Forms** ;



- Cliquez sur le bouton **OK** ;
- Une nouvelle r  f  rence    **System.Windows.Forms** appara  t dans la liste de r  f  rences du projet ;
- Dans l'Explorateur de Solutions, s  lectionnez le fichier **Program.cs** du projet **Coach** ;
- Faites un **clic droit** et s  lectionnez le menu **Afficher le code** ; L'  diteur de code Visual C# Express s'affiche sur la surface de travail ;

- En début de programme, ajoutez une directive **using**, qui indique que nous allons utiliser la librairie dans le code. Cela permet de rendre implicite les noms contenus par cet assemblage ;

Code C#

```
using System;
using System.Collections.Generic;
using System.Text;

using System.Windows.Forms;

namespace Coach
```

- A l'intérieur de la méthode **Main**, dans le bloc **catch**, modifier le code pour utiliser l'affichage dans une **boîte** de message ;

Code C#

```
catch (Exception)
{
    System.Console.
    WriteLine("Les arguments doivent être numériques");
    MessageBox.Show("Les arguments doivent être numériques");
}
```

- A l'intérieur de la méthode **Main**, dans le bloc **try**, modifier le code pour utiliser l'affichage dans une boîte de message, et en utilisant la classe string pour formater la sortie ;

Code C#

```
System.Console.
WriteLine("L'addition de {0} est : {1}.",
valeursAAfficher,
Calculateur.Ajouter(valeursACalculer));
MessageBox.Show(
    string.Format("L'addition de {0} est : {1}.",
        valeursAAfficher,
        Calculateur.Ajouter(valeursACalculer)
    )
);
```

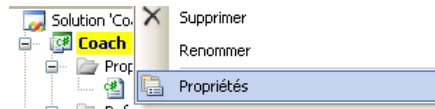
- Dans la barre d'outils de Visual C# Express, cliquez le bouton  pour sauvegarder les modifications réalisées ;
- Cliquez maintenant le bouton  dans la barre d'outils de Visual C# Express, pour lancer l'application. Les messages s'affichent maintenant dans une boîte de message.

2. Ajouter un icône de notification :

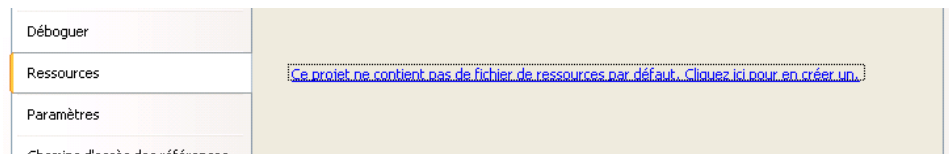


Dans le Framework, il y a beaucoup d'objets de présentation pour tout faire. L'objet à utiliser pour afficher un petit icône dans la barre des tâches en bas à droite de l'écran est le **NotifyIcon**.

- Dans Visual C# Express, sélectionnez le menu **Affichage > Autres fenêtres > explorateur d'objets** ;
- Localisez l'objet **NotifyIcon**, et cela tombe bien, il est dans la librairie **System.Windows.Forms** ;
- Faites un clic droit sur le projet **Coach** et en sélectionnez le menu **Propriétés** ;



- Sélectionnez l'onglet **Ressources** ;

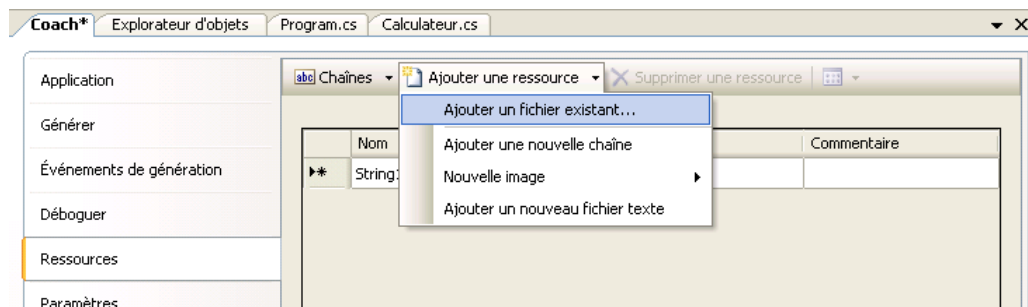


- Cliquez sur le lien **Ce projet ne contient ..** en bleu, afin de créer des ressources associées à ce projet.

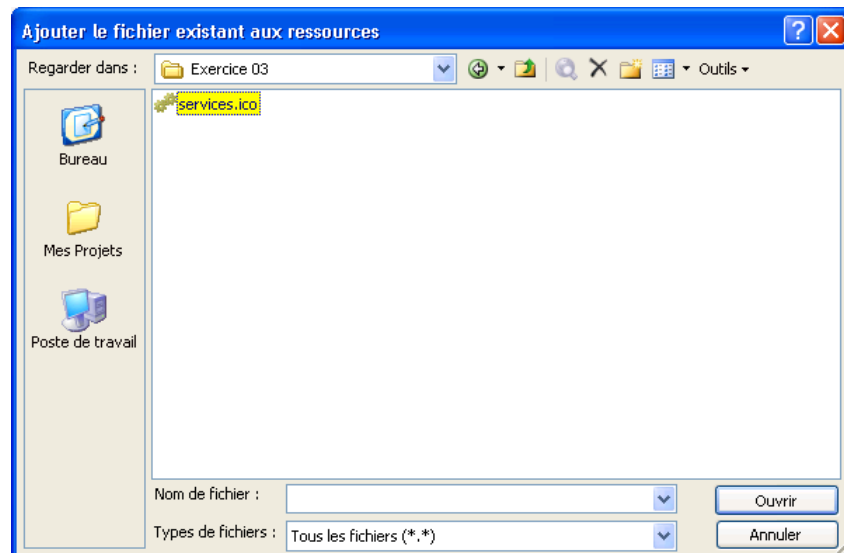


Les ressources du projet contiennent des chaînes de caractères, des images, des icônes, des fichiers audio, ou encore tout autre type de fichiers, qui vont être inclus à l'intérieur de l'assemblage pour être distribués avec le code compilé.

- Sur l'écran de gestion des ressources, cliquez le menu **Ajouter une ressource > ajouter un fichier existant...** ;



- Dans la boîte de dialogue **Ajouter un fichier existant aux ressources**, naviguez jusqu'au répertoire des fichiers utiles de l'exercice 03 de l'atelier 01 (par exemple **C:\Coach C#\Atelier 01\Fichiers Utiles\Exercice 03**) ;
- Sélectionnez le fichier **service.ico** et cliquez le bouton **Ouvrir** ;



- Dans la barre d'outils de Visual C# Express, cliquez le bouton  pour sauvegarder les modifications réalisées ;



Dans le projet **Coach**, un nouveau sous répertoire est apparu. C'est le dossier **Resources** qui contient le contenu des ressources comme les images et icônes. En outre, le sous répertoire **Properties** contient maintenant un nouveau fichier **Resources.resx**, qui contient la définition de toutes les ressources du projet.

- Dans l'Explorateur de Solutions, sélectionnez le fichier **Program.cs** du projet **Coach** ;
- Faites un **clic droit** et sélectionnez le menu **Afficher le code** ; L'éditeur de code Visual C# Express s'affiche sur la surface de travail ;
- A l'intérieur de la méthode **Main**, avant la définition des variables, ajoutez le code suivant pour définir un nouvel icône de notification ;

Code C#

```
static void Main(string[] args)
{
    NotifyIcon notifyIcon = new NotifyIcon();
    notifyIcon.Icon = Properties.Resources.services;
    notifyIcon.Visible = true;
    notifyIcon.Text = "Application de calcul";

    // Définition et assignation des variables locales
    string valeursAAfficher = string.Empty;
    double[] valeursACalculer = new double[args.Length];
}
```

- Dans la barre d'outil de Visual C# Express, cliquez le bouton  pour sauvegarder toutes les modifications réalisées ;
- Cliquez de nouveau le bouton  dans la barre d'outils de Visual C# Express, pour lancer l'application. Vous voyez apparaître votre icône dans la barre des tâches.



Il y a de nombreux objets fournis par le Framework 4.0. L'objectif de ce coach, au travers de ses différents ateliers, est de voir certains d'entre eux tout en se concentrant essentiellement sur les structures du langage.



Pour trouver de l'aide sur le contenu d'une librairie du Framework 4.0, il suffit de taper le lien suivant :

<http://msdn2.microsoft.com/<nom de la librairie>>

par exemple, pour tout savoir du contenu de la librairie **System.Xml**, tapez l'adresse suivante :

<http://msdn2.microsoft.com/system.xml>

Vous allez alors être redirigé vers la bonne page. Si ce n'est pas la bonne langue, changez dans l'url les paramètres de culture.

Par exemple :

<http://msdn2.microsoft.com/en-us/library/system.xml.aspx>

donne pour le français

<http://msdn2.microsoft.com/fr-fr/library/system.xml.aspx>

5 Pour aller plus loin...

5.1 Les liens utiles

Pour avoir plus d'information concernant les standards C# et .Net, voici quelques liens intéressants :

- Le standard ECMA concernant la spécification du langage C# :
<http://www.ecma-international.org/publications/standards/Ecma-334.htm>
- Le standard ECMA sur le Common Language Infrastructure (CLI) :
<http://www.ecma-international.org/publications/standards/Ecma-335.htm>
- Le standard ISO/IEC concernant la spécification du langage C# :
<http://www.iso.org/iso/en/CatalogueDetailPage.CatalogueDetail?CSNUMBER=42926&ICS1=35&ICS2=60&ICS3=&showrevision=y&scopelist=CATALOGUE>
- Le standard ISO/IEC sur le Common Language Infrastructure (CLI) :
<http://www.iso.org/iso/en/CatalogueDetailPage.CatalogueDetail?CSNUMBER=42927&ICS1=35&ICS2=60&ICS3=&showrevision=y&scopelist=CATALOGUE>

5.2 Idée/ressource

Si vous souhaitez apprendre la programmation d'application Web avec C#, il existe le Coach ASP.Net :

<http://www.microsoft.com/france/msdn/aspnet/coach/coach.aspx>

Et pour bien gérer vos projets de développement, suivez le coach Visual Studio Team System :

<http://www.microsoft.com/france/msdn/vstudio/teamsystem/evaluez/CoachVSTS.aspx>